Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Learning a Creator's Editing Style Online: Posterior Sampling With Style Priors, Uncertainty-Ordered Review, and Length Planning for Talking-Head Video

P. SUPREETH REDDY[1], B. VIJAYASIMHA REDDY[1], M. RUSHIVARUN REDDY[1], and GEETHA M[1]

[1]Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, Kerala, India (e-mail: am.sc.u4ae23142@am.students.amrita.edu, am.sc.u4aie23110@am.students.amrita.edu, am.sc.u4aie23128@am.students.amrita.edu, geetham@am.amrita.edu)

Corresponding author: P. Supreeth Reddy (e-mail: am.sc.u4ae23142@am.students.amrita.edu).

ABSTRACT A creator who films a talking-head Reel faces over a hundred small editing choices per clip, and two creators seldom make them the same way. One-click rules ignore that difference, and a supervised model needs labels a newcomer lacks. RoughCut treats each choice as an online reinforcement learning decision and takes its reward from what the creator already does: accepting, rejecting, or editing the transcript. A linear Thompson sampling agent proposes each edit. We add a tempered style posterior for the cold start, a review queue sorted by uncertainty, an exact posterior-sampling planner for target lengths, and randomized value iteration for visual pacing. On six simulated creators over ten seeds, retuned settings reached 93.1 percent acceptance by the tenth video, compared with 58.6 percent for a rule, and made 9 percent fewer wrong proposals than the best tuned neural agent. By the fiftieth video the networks had drawn level on acceptance. The planner cut held-out length error from 40.0 to 4.6 percent, costing 0.4 points of acceptance. Value iteration trailed a bandit on beat accuracy but earned better ratings when per-beat feedback was scarce. Three recorded clips tested the analysis pipeline; no formal human study has been run.

INDEX TERMS Contextual bandits, human-in-the-loop learning, off-policy evaluation, personalization, posterior sampling, reinforcement learning, Thompson sampling, video editing

TAKE a two-minute talking-head clip. The speaker restarts a sentence, pauses to think, and says "um" and "you know". Turning that into a Reel means removing dead air, choosing between takes, deciding which fillers matter, adding punch-in zooms, and highlighting a caption word now and then, more than a hundred choices in all. Many of them come down to taste: a storyteller keeps the pause before a punchline, and a creator chasing pace cuts it. No single correct edit exists for a universal model to learn.

Reinforcement learning (RL) [1] suits this because an agent learns from the outcome of its own actions and needs no list of right answers. It began with games [2] and now reaches recommendation [3]–[5], photo retouching [6], and the tuning of language models to human preference [7], [8]. When the reward comes from a person, as in the last two, every sample costs someone's time, so sample efficiency decides what is practical.

Editing adds its own constraints. A rejected pause suggestion says that one option was wrong and leaves the other three open. Creators judge a tool within a few videos, which means it must help from the first one. Taste drifts. A bad cut deletes a sentence, while a bad zoom merely looks odd. And "make this 30 seconds" applies to the whole edit, so no single cut can satisfy it.

We therefore set the following task. Given a raw clip of English speech and one creator's past corrections, propose every editing decision in the clip to match that creator, ask only about the likely mistakes, respect an optional target length, and update from whatever the creator does next.

One-click silence removal applies one threshold to everyone. Transcript editors [9]–[11] speed up cutting but leave

each decision to the person. A supervised model would need that person's labels first and would still not choose what to ask. In our setting labels arrive as a side effect of normal use, which favors RL with an agent that tracks its own uncertainty. Because most editing decisions barely influence later contexts, we model them with a contextual bandit [3], [12], which can learn from a few hundred interactions. The length budget gets an exact planner, and the pacing of visual effects gets a randomized value-iteration agent. We also measured whether deep RL [2], [13] really needs more data here, because the first version of that comparison turned out to be too kind to our method.

Our objectives were to build a simulation environment that exercises the real analysis code, to design a per-creator agent and a reward function covering both suggestions and manual edits, to outperform rule-based and generic baselines within a few videos, to keep learning stable across seeds, and to report where the method fails. This paper makes the following contributions.

- **A reward model in which ordinary editing is the training signal.** Word deletions, pause drags, and caption clicks are classified into overrides of learnable decisions, and a creator never performs a labeling step. In the ablation study, removing this mapping lowers early acceptance by 2.5 points.
- **A style posterior as a tempered bandit prior.** A Bayesian mixture over style prototypes supplies the prior mean of the bandit. Tempering lets the displayed style sharpen within about four interactions, whereas the prior moves slowly enough that one click cannot rewrite the policy. With leave-one-style-out priors, regret falls by 7 percent and the median number of decisions needed to reach 85 percent acceptance falls from 430 to 344.
- **Uncertainty-ordered review under a budget.** Thompson draws give each suggestion a confidence (expected calibration error 0.023), and that value sorts the review queue. At a 30 percent review budget, 86.9 percent of exported decisions are correct, against 78.4 percent for random order and 63.3 percent for timeline order.
- **Posterior sampling for a target-length edit with guarded reward.** We pose length fitting as a finite-horizon Markov decision process (MDP) whose reward is the learned bandit model, solve it exactly, and show that the history features must be frozen to stop the solver from exploiting model extrapolation. Length error on unseen videos falls from 40.0 to 4.6 percent.
- **A measured account of what exploration and sequential models add here.** Policies trained with and without exploration differ by about one point on unseen videos, while exploration costs up to 5.7 points during training. On visual pacing, randomized value iteration trails a bandit on per-beat accuracy but earns better whole-video ratings once per-beat feedback is scarce. Against fairly tuned networks, the linear method makes fewer mistakes over 50 videos, although the networks draw level on acceptance.

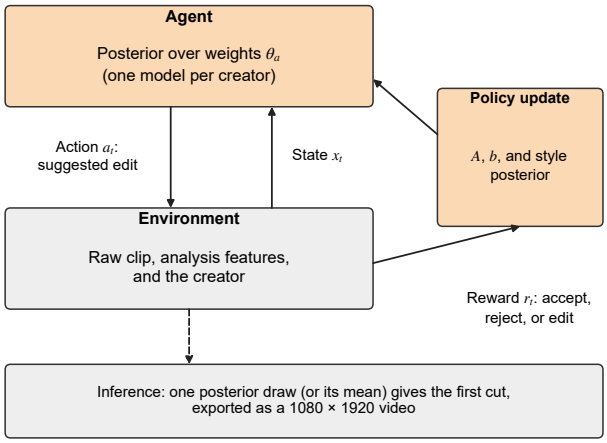


FIGURE 1. Conceptual learning loop. Training and inference share one path: each proposal is applied immediately and every correction updates the model before the next suggestion is scored.

Fig. 1 shows the interaction at the conceptual level. The creator is part of the environment, the state is the feature vector of the current decision, and the policy update happens after each piece of feedback rather than in a separate training phase.

I. RELATED WORK

A. COMPUTATIONAL AND TRANSCRIPT-BASED VIDEO EDITING

Aligning video to text has been the main route to faster editing. Interview footage can be cut from its transcript with jump cuts hidden by audio-visual cues [9], audio stories assembled with content-based tools [10], scripts linked to narration during authoring [14], and footage matched using the editor's own annotations in QuickCut [15]. The same alignment makes lectures skimmable [16] and drives B-roll recommendation in B-Script [11]. Film idioms can become cost terms for dialogue scenes [17], and a text edit can even resynthesize the talking head [18]. All of these save time, but their preferences are fixed idioms or settings the user supplies. None carries one user's corrections from project to project.

Language models changed the interface without changing that gap. LAVE [19] takes direction through an LLM agent, Write-A-Video [21] builds montages from text, and VideoDiff [20] offers alternative rough cuts to compare. A VideoDiff participant asked for a system that learned personal preferences over time, and that request motivates our work. Stating intent in words works well for structure and poorly for hundreds of small pacing choices.

B. REINFORCEMENT LEARNING FOR EDITING

Three adjacent systems show what RL has done near this task: video summarization with diversity and representativeness rewards [22], interpretable photo operations learned

with adversarial rewards [6], and, closest to ours, an editor trained on professional demonstrations with a pretrained vision-language model [23]. Each learns one policy offline for a general notion of quality. Our agent starts almost empty, trains on one person during normal use, and is scored on matching that person.

C. CONTEXTUAL BANDITS AND THOMPSON SAMPLING

LinUCB made contextual bandits a standard tool for personalization through news recommendation [3], with analysis for linear payoffs [12]. Thompson sampling is far older [24] and has strong empirical and theoretical support [25]–[28]. Neural versions replace the linear model with a network [30], yet simple linear posteriors on good features remain hard to beat [29]. We saw the same over the first videos, and we also saw networks close the gap as data accumulated. Surveys cover the applications [31] and the theory [32], and discounted estimators handle non-stationary preferences [33]. We reuse these parts directly. Our contribution lies in the editing-specific structure built around them.

D. PRIORS ACROSS USERS

Sharing information across tasks is the usual answer to a user's cold start. Meta-Thompson sampling learns a prior over tasks [34], other work biases linear bandits toward a learned mean [35], and latent bandits assume each user belongs to one of a few hidden types [36]. Our style posterior is closest to latent bandits, with one difference: its mixture only sets a prior mean for a per-creator ridge model, so the agent can leave all the prototypes behind when a creator matches none. We test that case explicitly.

E. LEARNING FROM HUMAN FEEDBACK

TAMER [37] showed agents learning from human evaluative signals, which were later found to depend on the agent's current policy [38]. Preference-based deep RL [7] and RLHF for language models [8] learn a reward model from comparisons. Recommender research treats usage as weak evidence [39], and we copy that caution: a suggestion seen and left alone earns half the reward of an explicit accept, and one never seen earns nothing. Our review queue applies the idea behind active learning [40], with bandit uncertainty deciding what to ask.

F. PLANNING AND MODEL EXPLOITATION

Posterior sampling for RL (PSRL) [41] extends Thompson sampling to whole episodes by solving an MDP sampled from the posterior. Learned models invite exploitation of their errors, which led to limited rollouts [44] and uncertainty penalties [45], and constrained MDPs treat budgets formally [46]. Our length planner uses PSRL with a known transition model, and we report a failure in which the solver exploited a linear reward until it cut an entire video. We first planned to use proximal policy optimization [13], then dropped it: with known deterministic transitions, an exact solution is cheaper and optimal for the given reward. Pacing does depend on state. There we use randomized least-squares value iteration [42], [43], the closest sequential relative of our Thompson sampling.

TABLE 1. Qualitative Comparison With Existing Approaches

Approach	Personal	Online	Queue	Length
Silence-removal rules	No	No	No	No
Transcript editors [9], [10]	Manual	No	No	Manual
LLM-directed editors [19], [20]	By prompt	No	No	By prompt
RL from demonstrations [23]	No	No	No	No
Bandit with no style prior (baseline)	Yes	Yes	No	No
Proposed	**Yes**	**Yes**	**Yes**	**Yes**

Personal: adapts to one user. Online: improves during use. Queue: chooses what to ask about. Length: plans to a target duration.

G. OFF-POLICY EVALUATION

Offline evaluation estimates from logs what a different policy would have achieved [47], [48]. The main tools are inverse propensity scoring [49], its self-normalized form [50], and doubly robust estimators [51], with separate estimators for slates [52]. Adaptive logging complicates inference [53], and bootstraps with few clusters come out too narrow [54]. We log a propensity with every suggestion and check the estimators against simulator truth before using them on real logs.

H. SPEECH ANALYSIS FOR DISFLUENCIES

Whisper [55] transcribes accurately but tidies disfluent speech, which has prompted work on prompts for verbatim output [56]. Forced alignment with a character model such as wav2vec 2.0 [57], through connectionist temporal classification (CTC) [58], gives accurate word timing, as WhisperX [59] demonstrated. Disfluency detection has used sequence models [60], joint parsing [61], and end-to-end systems [62]. These treat removal as uniform clean-up, whereas our creators disagree about which fillers to keep.

I. WHY A DIFFERENT APPROACH IS NEEDED

Table 1 summarizes the gaps. Our search was not exhaustive, but we found no system that learns one creator's preferences online for this class of edit, plans to a length target, and logs what counterfactual evaluation needs. The proposed design combines all three with models small enough to run per user on a laptop.

II. SOLUTION APPROACH

RoughCut turns raw speech footage into a set of editing decisions, proposes an action for each with per-creator agents, and applies every proposal at once. The creator therefore starts from a finished first cut. Corrections then flow back through a reward mapping. The bandit itself is standard. The new parts surround it: a style posterior as its prior, a review queue built from its uncertainty, an exact planner for target lengths, a value-iteration agent for pacing, and propensity

logging for counterfactual evaluation. Fig. 2 shows the full pipeline.

A. ENVIRONMENT DESIGN

1) Units and Analysis

Audio is transcribed with faster-whisper running the small English Whisper model [55] at beam size 10, without conditioning on previous text because conditioning caused looping output over silence. The decoder runs in its own process: inside a process that had already prepared the footage, identical audio produced 305 words instead of 324 on one clip and lost a whole sentence. We did not find the internal cause, but three repeated runs in fresh processes were identical. A short disfluency-rich prompt nudges the decoder to keep fillers. Words with no voiced frames beneath them are discarded as hallucinations. Word boundaries are then re-derived by Viterbi alignment of Whisper's characters to the frame-level CTC output of wav2vec 2.0 [57], following WhisperX [59], since Whisper's own timestamps stretch words across pauses.

Two steps go beyond that standard recipe. First, the CTC model's own greedy reading is compared with Whisper's word list using sequence matching, and only two kinds of extra token are trusted: a hesitation sound lasting at least 80 ms, or an exact repeat of a neighboring word. This restored the "I I" stutter in our test clip that Whisper had written as one word. Everything else in the greedy reading is ignored, as its spelling is poor. Second, a filler touching its neighbors is cut only if the audio has a dip at both edges. We search within 80 ms of the boundary for a frame at least 6 dB quieter than the speech on either side and move the boundary onto it. A boundary with neither gap nor dip stays, because cutting there clips a neighboring word. Spoken lines can also be aimed at the wrong audience. "This one is for YouTube Shorts", said to a colleague, is clean, complete English and passes filler and retake detection, so each sentence also gets a crew score. Words about production ("voiceover", "one for Instagram") add 0.4 and softer signals add 0.2. Quiet delivery, isolation between long gaps, and shortness count only when such words are present. Viewer-directed words ("comment", "subscribe") subtract from the score, and at 0.5 or more the first edit cuts the line, which the creator can restore like any other cut. We set these weights by hand from very few clips and treat the rule as a useful default, not a validated detector.

The clip is then segmented into four kinds of unit: phrases (split at gaps of at least 0.25 s and at sentence ends, capped near 6 s), pauses (gaps of at least 0.15 s), filler tokens, and caption lines of at most four words. Retakes receive particular care, as they decide which sentences survive. A fuzzy token match catches content retakes. Because most false starts consist of function words ("you'll also be,"), a separate matcher compares all words of a phrase with the speech that follows it, even across phrase boundaries. A complete earlier delivery beats a later repeat only if it is clearly better spoken, with a margin of 0.08 on a score that penalizes fillers and restarts. Phrase embeddings from a sentence transformer [63] provide redundancy and topic-centrality scores, and YuNet [64] tracks the face for a 9:16 crop. All of this runs once, at upload.

TABLE 2. Decision Types, Actions, and Context Features

Type	Actions	d	Representative features
Phrase	KEEP, CUT	39	duration, retake score, earlier take, incomplete, filler ratio, centrality, redundancy
Pause	KEEP, TIGHT_40, TIGHT_15, REMOVE	21	gap (log and bins), sentence end, voiced ratio, take boundary
Zoom	NONE, Z115, Z130	27	loudness, punctuation, face size, time since last zoom
Filler	KEEP, REMOVE	23	token one-hot, position, gaps, verb-like use of "like"
Emphasis	NONE, or one of up to 3 words	16	per word: number, proper noun, TF-IDF, stress, negation
Pace	S100, S110, S125	18	speech rate, duration, loudness, filler ratio
Music	FROM_START, AFTER_INTRO	7	video length, first-sentence length, type
Beat	NONE, Z115, Z130, TEXT_POP, SFX	29	time since last beat, its kind, streak, centrality
Title	one of up to 3 lines	8	first line, question, number, "you", centrality

2) State Space

Each decision has a context vector $x_t \in \mathbb{R}^d$, normalized and partly binned, with d between 16 and 39 depending on its type (Table 2). Contexts also carry a one-hot video type and a bias term. Because decisions are processed left to right, the context can include history features such as the ratio of output time to source time so far and the time since the last visual change. In the bandit layer these features are the only way an earlier action affects a later state.

3) Action Space and Dynamics

Phrase, pause, filler, pace, and music decisions use disjoint arms, with one weight vector per action. Caption emphasis uses a shared weight vector over per-word features, and what is learned about numbers on one line transfers to all lines. Pause actions leave 0.40 s, 0.15 s, or 0.03 s of silence, or keep the gap untouched. All decisions compile into a single edit decision list, the only representation that both the browser preview and the FFmpeg renderer read. Speech edges are padded by 40 ms, every cut has an 8 ms audio fade, and clip edges snap to the 30 frames per second grid so that the rendered length equals the planned length. The transition dynamics are deterministic and known: applying an action changes only the running output time and history counters. The simulator experiments use a zoom decision with the first three beat actions; the deployed editor replaces it with the beat decision of Section II-C4.

4) Reward Design

Table 3 gives the reward mapping. The reward is defined over the creator's reactions and not over an aesthetic score, so "good" simply means "this creator would keep it". A binary

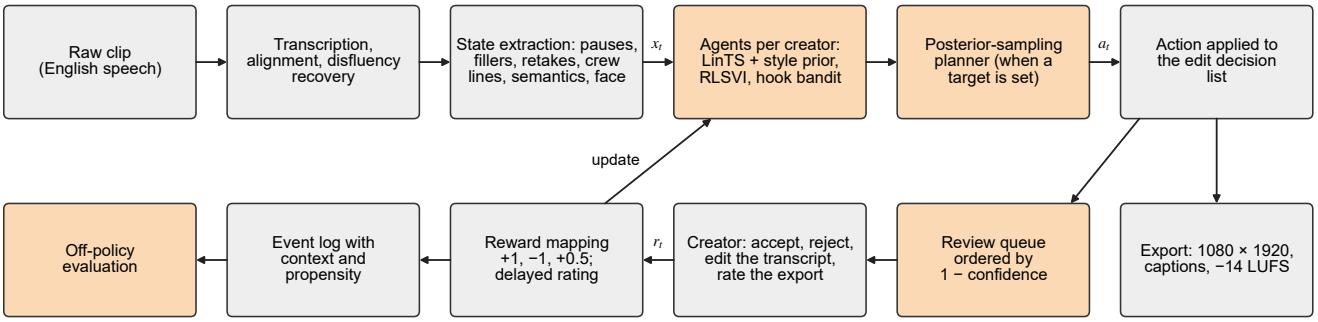


FIGURE 2. Complete pipeline. Orange blocks are the learning components. The top row runs when a clip is uploaded. The bottom row runs each time the creator reacts, and policy state can always be rebuilt by replaying the event log, which is how undo works.

TABLE 3. Reward Mapping for Suggestions and Manual Edits

Creator's reaction	Bandit update
Accept	proposed arm $r = +1$
Reject, binary decision	proposed $r = -1$; other arm $r = +1$
Reject, multi-arm decision	proposed $r = -1$ only
Override to a' (panel, transcript, or timeline)	proposed $r = -1$; a' $r = +1$
Untouched at export, seen	proposed $r = +0.5$
Untouched, never seen	no update
Rating at export, $\rho \in \{-1, 0, 1\}$	rhythm agent only (Section II-C4)
Undo or redo	flip event flag, replay log

rejection implies that the other arm was right, and a multi-arm rejection gives only negative evidence, which is why exploration could matter there. Manual edits use the same mapping: deleting at least 70 percent of a phrase counts as a phrase override, whereas a smaller word-level cut is applied but not learned from, because it lies outside the action space. The player tracks which units were seen, and an untouched suggestion only earns the weak implicit reward if the creator could actually have noticed it.

5) A Worked Example

One of our recordings shows how these parts meet. The speaker says "You'll also be learning about what's the difference between green screen and", stops, tries the line several more times ("you'll also be," and so on), and finally delivers "you'll also be learning the difference between green screen and blue screen." The content-token matcher finds little here because the fragments are almost entirely function words. The all-words matcher finds each fragment again at the start of the later speech and gives it `is_earlier_take` $= 1$ and a high retake score. These two features enter the phrase context, and a learned weight on them decides the edit, so a creator could in principle teach the agent to keep such a passage. A new creator's first cut of this recording removes every fragment. The surrounding pauses merge into one cut, and the viewer hears "and so much more." followed directly by the clean take, with an 8 ms fade at the join.

B. RL ALGORITHM

We chose linear Thompson sampling over PPO or deep Q-learning for three reasons. A creator supplies hundreds of signals, not millions. The linear model's weights can be read back to the creator as an explanation and a profile. Its closed-form posterior also makes undo exact, since replaying the log yields the same state. A closed-form ridge step after every event replaces any learning rate or batch size.

For arm a of a decision type, the expected reward is modeled as $\theta_a^\top x$. Prior and data statistics are held separately:

$$A_a = \lambda I + A_a^d, \qquad \hat{\theta}_a = A_a^{-1}\left(\lambda\,\theta_a^0 + b_a^d\right), \tag{1}$$

$$A_a^d \leftarrow \gamma A_a^d + w\,xx^\top, \qquad b_a^d \leftarrow \gamma b_a^d + w\,r\,x, \tag{2}$$

where λ is the prior strength, θ_a^0 the prior mean, γ the discount, and w an event weight. To act, the agent draws $\tilde{\theta}_a \sim \mathcal{N}(\hat{\theta}_a, v^2 A_a^{-1})$ and takes

$$a_t = \arg\max_a \tilde{\theta}_a^\top x_t, \tag{3}$$

replaced by a uniform arm with probability ϵ. The action probabilities are estimated from $M = 200$ draws,

$$p(a \mid x) = (1-\epsilon)\,\frac{1}{M}\sum_{m=1}^{M} \mathbf{1}\left[a = \arg\max_{a'} \tilde{\theta}_{a'}^{(m)\top} x\right] + \frac{\epsilon}{K}, \tag{4}$$

and $p(a_t \mid x_t)$ is stored as both the logged propensity and the confidence of the suggestion. The goal is to minimize regret against the creator's own ideal action a_t^*,

$$R_T = \sum_{t=1}^{T} \mathbf{1}\left[a_t \neq a_t^*\right], \tag{5}$$

which counts wrong proposals and is what a creator pays in corrections.

Replay reproduces the online posterior because the simulator, the application programming interface, and undo all apply events through a single learner object, and we test that property directly. Each suggestion also stores its raw context, so a change of prior or a schema migration can rebuild the sufficient statistics without losing what a creator has taught the agent.

Each suggestion can also be explained. For the proposed action and its runner-up, the difference of scores decomposes into per-feature contributions $(\hat{\theta}_a - \hat{\theta}_{a'})_j \, x_j$. The "Why?" panel turns the three largest into a sentence, and the largest weights across all decisions form the creator's profile page. These statements describe the model, not the creator's reasoning, and we have not studied whether people find them useful.

C. NOVEL COMPONENTS

1) Tempered Style Posterior as Prior

For K style prototypes $\theta^{(k)}$, ridge-fitted offline, and a pooled generic prior, every event updates a log weight,

$$\log w_k \leftarrow \log w_k - \frac{\left(r - \theta_a^{(k)\top} x\right)^2}{2\sigma^2}. \tag{6}$$

With $\sigma = 0.7$, the posterior concentrates after a few events and drives the sentence shown to the creator. The bandit, however, uses weights tempered by $(\sigma/\sigma_p)^2$ with $\sigma_p = 3$ to form its prior mean $\theta^0 = \sum_k \tilde{w}_k \theta^{(k)}$. Tempering separates two quantities that a plain latent-bandit posterior ties together: what we say about the creator and how far one click can move the policy. Without it, one rejected KEEP flipped 65 percent of kept phrases on the next video at $\lambda = 10$. With a stronger keep/cut prior of $\lambda = 30$ that figure fell to 8 percent. The same safety argument removes the ϵ floor on keep/cut in the product, since a uniform draw there can delete a sentence, and Thompson draws still explore where the posterior is unsure.

2) Uncertainty-Ordered Review

Suggestions are sorted by $1 - p(a_t \mid x_t)$, and those with confidence below 0.7 form the "needs your eye" list. Everything else stays applied and available for a spot check. After each correction, unreviewed suggestions are re-scored, with two guards: an action is changed only if Thompson sampling would pick it at least 80 percent of the time, and one nudge moves at most a quarter of a decision type. Fig. 3 shows the resulting interface.

3) Target-Length Planning by Posterior Sampling

A duration target cannot be enforced one decision at a time. We define an MDP over the phrase and pause decisions in time order. Its state is $s = (o, c)$, with output seconds o on a 0.1 s grid and a flag c for whether the previous phrase was kept. The reward at stage k is the learned model, and the terminal value penalizes missing the target T:

$$r_k(a) = \theta_a^\top x_k, \tag{7}$$

$$V_K(o) = -\beta \max\left(0, \tfrac{o}{T} - 1 - \tau\right) - \beta_u \max\left(0, 1 - \tau - \tfrac{o}{T}\right), \tag{8}$$

with $\beta = 150$, $\beta_u = 6$, and $\tau = 0.05$, so a target works mainly as a limit and a clip whose good edit is shorter is not padded. Backward induction,

$$V_k(s) = \max_a \left[r_k(a) + V_{k+1}\left(s'(s,a)\right)\right], \tag{9}$$

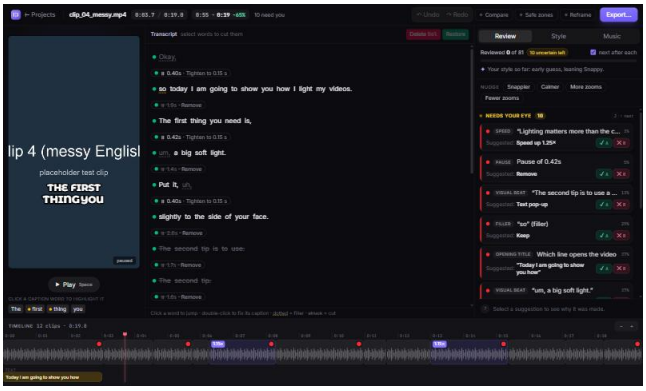


FIGURE 3. Editor on a synthetic test clip: captioned preview, transcript with struck-out cuts, review queue sorted by uncertainty, and a timeline with confidence dots.

gives the exact optimum, because transitions are known. A phrase that continues the sentence of the previous phrase must share its fate, so a length target can never strand half a sentence.

Exploration follows PSRL [41]: once per video, θ is drawn from the bandit posterior and the MDP is solved for that draw. The plan is closed loop: a creator's override simply moves the state to another row of the solved table. Propensities are the share of 16 further posterior draws whose optimal plan takes the same action at that state. Since the planner's step model only approximates the rendered edit, the planned target is corrected against the real edit length, up to twice per video.

How to treat history features turned out to be the key design decision. In our first version they followed the MDP state, and the result looked exact but was badly wrong: on simulated test videos the solver cut every phrase, because the linear model had never seen a state like "everything cut so far" and extrapolated high reward there. Model-based RL calls this model exploitation [44], [45], and here it appears in a small, exactly solvable form. We now hold those features at their values along the bandit's own posterior-mean trajectory, keeping the reward inside the data's support. As a result the reward ignores the plan's own history, and only the length budget and the sentence ties couple the decisions.

For K stages, n buckets of output time, and at most four actions, backward induction takes $O(K\,n)$ table operations per solve, vectorized over the grid. A typical video has about 65 stages and a few hundred buckets. One complete planning call runs up to three solves for calibration and 17 more for the acting plan and the propensity draws, finishing in about 0.2 s on a laptop CPU (Table 9). A sampled policy-gradient method would need far more computation to approximate the same table and would be harder to explain.

4) Visual Rhythm by Randomized Value Iteration

This decision is sequential by nature, because whether a punch-in works depends on what came before it. At each kept phrase longer than 1 s, the agent chooses among no effect, a 1.15x or 1.3x zoom, a text pop-up, and a sound

Algorithm 1 Visual Rhythm Agent (RLSVI)

Require: Prior θ^0, logged episodes $\mathcal{D}$
for each new video **do**
  Perturb prior: $\tilde{\theta}^0 \leftarrow \theta^0 + \sigma\lambda^{-1/2}\varepsilon$
  Perturb rewards in $\mathcal{D}$; add the rating to steps
  for 10 sweeps **do**
    Compute targets by (10); update θ_a by (11)
  end for
  for each kept phrase in time order **do**
    Observe $\phi(s)$; take $a = \arg\max_a \theta_a^\top \phi(s)$
    Observe the creator's reaction; advance the state
  end for
  Observe the rating ρ; append the episode to $\mathcal{D}$
end for

effect. The state $\phi(s) \in \mathbb{R}^{29}$ holds binned time since the last beat, the kind of that beat, the streak length, beats per minute so far, phrase features, and products of time bins with topic centrality, because the interesting rules are conjunctions. Unlike the planner's frozen features, these are true state: the chosen action changes the next ϕ.

With a linear $Q(s,a) = \theta_a^\top \phi(s)$, RLSVI [42] perturbs each reward with Gaussian noise and each prior with a draw, and then runs fitted value iteration on the perturbed data,

$$y_i = r_i + \xi_i + \gamma \max_{a'} \theta_{a'}^\top \phi(s_i'), \qquad \xi_i \sim \mathcal{N}(0, \sigma^2/w_i), \quad (10)$$

$$\theta_a \leftarrow \Big(\sum_{i:a_i=a} w_i \phi_i \phi_i^\top + \lambda I \Big)^{-1} \Big(\sum_{i:a_i=a} w_i \phi_i y_i + \lambda \tilde{\theta}_a^0 \Big), \quad (11)$$

with $\gamma = 0.9$, $\sigma = 0.15$, $\lambda = 1$, and 10 sweeps. The agent then acts greedily on that one draw for the whole video, which is Thompson sampling extended to an episode. The prior mean θ^0 is fitted to a calm hand-written rule, one modest beat after several quiet seconds. The rating at export is delayed feedback: it adds 2ρ to the last step, for value iteration to carry backward, and 0.5ρ to every step. Those two weights were chosen on tuning seeds only. Algorithm 1 gives the procedure, and setting $\gamma = 0$ turns the same code into a bandit on the same features, which is the baseline we compare with.

5) Opening Title, Pace, and Music Timing

Three smaller decisions use the same machinery. A hook bandit picks the on-screen title from up to three candidate lines, using a shared Thompson model over line features, so a preference for questions carries across videos. It does not move footage to the front. Phrase speed and music timing are ordinary LinTS decisions with a fixed bonus on the plain option, so an untrained agent stays at normal speed. Cosmetic decisions are capped in the review queue, since they are numerous and cheap to get wrong. These three decisions were not evaluated in the simulator and count here as product features, not research results.

6) Counterfactual Evaluation

Each logged suggestion stores (x_i, a_i, p_i, r_i). For a target policy π, with weight $w_i = \pi(a_i \mid x_i)/p_i$ and a cross-fitted ridge reward model $\hat{q}$, we compute the direct method (DM), inverse propensity scoring (IPS), self-normalized IPS (SNIPS), and doubly robust (DR) estimates, plus self-normalized DR (SNDR):

$$\hat{V}_{\mathrm{DR}} = \frac{1}{n} \sum_{i=1}^{n} \Big[\sum_a \pi(a \mid x_i) \hat{q}(x_i, a) + w_i \big(r_i - \hat{q}(x_i, a_i) \big) \Big]. \quad (12)$$

The bootstrap resamples whole videos because decisions within a video are dependent. With few videos, percentile intervals were too narrow [54]. We therefore report the estimate $\pm\, t_{G-1,0.975} \sqrt{G/(G-1)}$ times the bootstrap standard deviation, for G videos.

Algorithm 2 Online Editing Agent for One Creator

Load prototypes; set $\log w_k \leftarrow 0$, $A^d, b^d \leftarrow 0$
Set prior mean θ^0 from tempered w
for each uploaded video **do**
  Analyze the clip into units and features
  if a target length is set **then**
    Solve the planner (Algorithm 3)
  end if
  for each decision t in time order **do**
    Observe context x_t (static features and history)
    $a_t \leftarrow$ planner action if planned, otherwise (3)
    Compute and log $p(a_t \mid x_t)$ by (4); apply a_t
  end for
  Sort the review queue by $1 - p(a_t \mid x_t)$
  while the creator is editing **do**
    Observe an event; map it to updates (Table 3)
    Update A^d, b^d by (2) and w by (6)
    Recompute θ^0; re-score unreviewed suggestions
    On undo: flag the event and replay the log
  end while
  At export: give seen, untouched suggestions $r = +0.5$
end for

Algorithm 3 Target-Length Planner with Posterior Sampling

Require: Stages with frozen history features, target T, posterior
Solve (9) with posterior means $\hat{\theta}$
for up to two calibration rounds **do**
  Roll out the plan and measure the real edit length L
  if $|L - T|/T \le \tau/2$ **then** stop
  Shift the planned target by $T - L$ and solve again
end for
Draw $\tilde{\theta} \sim$ posterior; solve (9): acting plan
Solve for 16 more draws: propensity plans
for each stage reached, at the state (o, c) actually observed **do**
  Act by table look-up in the acting plan
  Propensity $\leftarrow$ share of draws with the same action
end for

D. ALGORITHM DESCRIPTION

Algorithm 2 gives the online loop, Algorithm 3 the planner, and Algorithm 1 the rhythm agent. There is no stopping condition during use, because the agent learns for as long as the creator keeps editing, with style weights frozen after 200 events. Each simulated experiment stops after a fixed number of videos.

E. SYSTEM IMPLEMENTATION

The method is deployed in a complete editor so that feedback comes from realistic use. The backend is written with FastAPI and stores projects, suggestions, and events in SQLite. The frontend, in React and TypeScript, renders the edit decision list in the browser with skip ranges, CSS transforms for crop and zoom, and HTML captions, with no server render while editing. Export uses an FFmpeg filter graph that crops from the original source, burns in captions, applies high-pass filtering, and normalizes loudness to -14 LUFS over two passes. Optional background music is ducked under speech with sidechain compression.

Batch uploads use the posterior mean without exploration. A video is marked ready to post when the sum of $1-$confidence over its suggestions is below 3 and the creator's recent acceptance rate is at least 0.85. One-click export from that state deliberately produces no training signal, because nobody reviewed anything. The 317 automated backend tests cover the reward mapping, replay, and timing rules, and a further 166 cover the interface. They verify the implementation and do not validate the method.

III. EXPERIMENTAL RESULTS

A. EXPERIMENTAL SETUP

1) Environment and Data

No offline dataset of real creators' corrections exists for this task and we have not yet collected our own, so the main evaluation uses a simulator. It generates timed transcripts with planted mistakes: abandoned takes followed by retakes, tangents, fillers, long pauses, and joins between takes. These pass through the same segmentation, filler, pause, and feature code as real footage, giving the agent the same state representation, with roughly 160 decisions per video.

Simulated creators are six personas with hidden rules for every decision type (Table 4). A persona reviews a proposal, gives a label that is wrong with probability 0.05, and corrects a disagreement with probability 0.6. Half of these corrections are made as manual edits passing through the real classification code. All priors are fitted without the persona under test (leave-one-style-out), on training videos disjoint from the evaluation videos. A persona's own prototype is never available to it. We also use three recorded clips for the analysis layer.

2) Hardware and Software

All runs used one Windows 11 laptop with an Intel processor reporting 16 logical cores and no GPU. The software was Python 3.10, NumPy 2.2, SciPy 1.15, faster-whisper 1.2, PyTorch 2.14 (CPU), Transformers 5.18, Sentence-Transformers 6.1, OpenCV 5.0, and FFmpeg 9.0. Agents are our own implementation, and the deep baselines are written directly in PyTorch. Gymnasium and Stable Baselines3 are not used, because the environment runs the product's own decision code.

TABLE 4. Simulated Creators (Hidden Ground Truth)

Persona	Pauses	Phrases, fillers, zoom
Snappy	remove over 0.5 s, else 0.15 s	cut fillery or garbled; remove all fillers; zoom on loud lines
Relaxed	keep under 0.8 s, else 0.40 s	cut earlier takes; remove um, uh; no zoom
Storyteller	0.40 s after sentence end, else remove	keep story; remove um, uh, you know; zoom on ? and !
Minimalist	remove all	keep most central up to 30 s; no zoom
Focused	all to 0.15 s	cut off-topic and repeats; zoom on central lines
Blend	as Snappy	phrases and zooms as Storyteller

TABLE 5. Hyperparameters

Parameter	Simulator	Product
Prior strength λ	2	10 (keep/cut 30)
Posterior scale v	0.25	0.15
Discount γ	0.995	0.995
Exploration floor ϵ	0.02	0.005 (keep/cut 0)
Thompson draws M	200	200
Style width σ / prior width σ_p	0.7 / none	0.7 / 3
Implicit reward	0.5	0.5
Style frozen after	200 events	200 events
Planner β, β_u, τ, grid, draws	150, 6, 0.05, 0.1 s, 16	
Episodes per run / seeds per persona	10 / 10	
RLSVI γ, σ, λ, sweeps	0.9, 0.15, 1, 10	
Retuned λ, v, ϵ (Section III-D1)	2, 0.15, 0	
Neural: hidden, batch, lr, ϵ, steps per event	64–64, 32, 3×10^{-4}, 0, 4	
NeuralLinear: lr, v, refit interval	10^{-3}, 0.1, 32	

3) Hyperparameters

Table 5 lists the settings. For the linear agents, the terms learning rate and batch size have no direct counterpart. The update of (2) is closed-form and applies per event, so the closest analogs are λ and γ. An episode is one video and an environment step is one decision. Unless noted, results use ten seeds per persona, or 60 runs per method, and ten videos per run. Intervals are 95 percent t intervals over runs.

B. PERFORMANCE EVALUATION

1) Metrics

Acceptance is the share of proposals equal to the creator's ideal action. Return per decision is the mean realized reward from Table 3. We use cumulative regret from (5). Success, the task-level condition, means that a video reaches at least 85 percent acceptance, measured over videos 6 to 10. Steps to target is the number of decisions until a run first completes a video at or above 85 percent. Corrections per minute of footage measure the effort that remains. For target lengths we report relative length error measured on the real edit, and for review the share of correct decisions in the exported edit. Together these are the problem-specific metrics for this domain.

2) Learning and Regret

Return rises steeply over the first 500 decisions, about three videos, and then flattens (Fig. 4). Both fixed baselines stay flat, and the learners separate from them on the first video. By Table 6, the proposed method ends at 91.0 percent acceptance with 181.5 wrong proposals in 10 videos, against 668.9 for Rule and 780.1 for Global.

3) Acceptance by Decision Type

Pauses show the widest gap across the five decision types (Fig. 5). Rules score only 0.42 there on video 10 because the personas disagree most, and learning reaches 0.96. Keep/cut starts high for every method because most phrases should be kept, and Rule's 0.87 is close to the learners. Emphasis is the hardest learned decision at 0.88, since our features only partly explain the choice of word.

Per creator (Table 7), the proposed method leads Rule on all six, by 19 to 50 points. The gap is smallest for Blend and Snappy, whose pause rule resembles Rule's, and largest for Relaxed and Minimalist, who are poorly served by cutting everything over half a second. The style prior helps most on Snappy and Blend relative to plain LinTS. On Minimalist and Focused it falls slightly behind (0.857 against 0.870, and 0.915 against 0.929), so the mean gain is not uniform.

Table 8 breaks the results down by decision type. The product's stronger keep/cut prior gives up about three points of keep/cut acceptance (0.876 against 0.905) to prevent sudden large changes, and it recovers the difference on pauses and zooms. We accept this trade because a wrong CUT removes what someone said, while a wrong pause length is hardly noticed.

4) Stability Across Seeds

The standard deviation of per-seed mean acceptance at video 10 is 0.013 for the proposed method and 0.018 for the plain bandit, both small compared with the 0.32 gap from Rule. Most of the spread across all runs (0.045) comes from differences among personas and not from randomness, since Relaxed reaches 0.974 and Minimalist only 0.857. Minimalist's rule depends on a per-video ranking of centrality that a linear model cannot fully represent.

5) Computational Efficiency

Table 9 shows the learner's cost, measured on one thread. A suggestion takes well under 1 ms on average, and waiting time depends on media analysis, not learning. The bandit has 305 parameters per creator and its data statistics occupy 67 KiB. The rhythm agent adds 145 parameters and takes 16 ms to prepare a video after ten videos of history. A neural bandit run with one gradient step per event took 20 s against 2.4 s for LinTS. Analysis took 98 s for the 54.5 s clip, 119 s for the 44.2 s clip, and 270 s for the 238.5 s clip, measured while the test suite was running on the same machine; these are upper figures.

C. RESULTS AND DISCUSSION

1) Does the Style Prior Fix the Cold Start?

On videos 1 to 3, acceptance is 0.824 with no prior, 0.831 with a generic prior, and 0.845 with the style mixture. This modest gain fades as personal data accumulates, and by video 6 the three curves lie within one point of each other (0.889, 0.889, 0.899). The prior mainly saves time: the median number of decisions to reach an 85 percent video drops from 430 to 344, about half a video sooner. The style itself is identified (weight above 0.8) after a median of 3.5 events, although Snappy takes 7.5 and at most 21 because it overlaps with Blend. These figures use leave-one-style-out priors and probably describe a new creator somewhat similar to the known styles. A person who resembles none of them should expect less.

The style mixture's lead peaks on the first video (0.806 against 0.767 with no prior) and is nearly gone by the fourth, where all three sit near 0.885 (Fig. 6). A paired test on the first three videos gives a gain of 1.5 points over the generic prior ($p = 0.004$), or about 13 fewer wrong proposals across ten videos, for one mixture update per event.

2) Is the Policy Personalized?

To test personalization, we trained on one persona and evaluated the frozen policy on each (Fig. 7). The diagonal averages 0.936 and the off-diagonal 0.468, the largest effect in this paper: a policy excellent for one creator is close to useless for another. Blend and Snappy reach about 0.85 to 0.87 on each other because they share pause, filler, and emphasis rules by construction. The result supports the premise that one global model cannot serve everyone, with the caveat that we wrote the personas to differ.

3) Can It Follow Changing Taste?

When a creator switches from Relaxed to Snappy at video 6 (Fig. 8), acceptance collapses to 0.54 with no discounting, 0.61 with $\gamma = 0.995$, and 0.68 with $\gamma = 0.98$, and all three recover above 0.8 within three to four videos. Stronger discounting softens the drop at a price: lower acceptance before the switch (0.935 against 0.950) and at the end of the run (0.834 against 0.871). The ablation shows the same cost under stationary conditions, and we treat $\gamma = 0.995$ as a compromise.

4) Does Length Need Planning?

We trained the bandit and the bandit with planner online, then froze both on four unseen videos, for targets of 20, 30, and 45 s (Fig. 9). The bandit's length error remains 40.0 percent after six videos because nothing in a per-decision policy knows the target. With the planner the error ends at 4.6 percent, below the 5 percent tolerance, and stays small from the first video since planning enforces the target. Acceptance falls by 0.4 points (0.879 against 0.883), within the intervals. Keep/cut acceptance stays near 0.68 for both, so neither reproduces Minimalist's choice of phrases. Planning fixes the length but not the selection, the method's clearest unsolved problem.

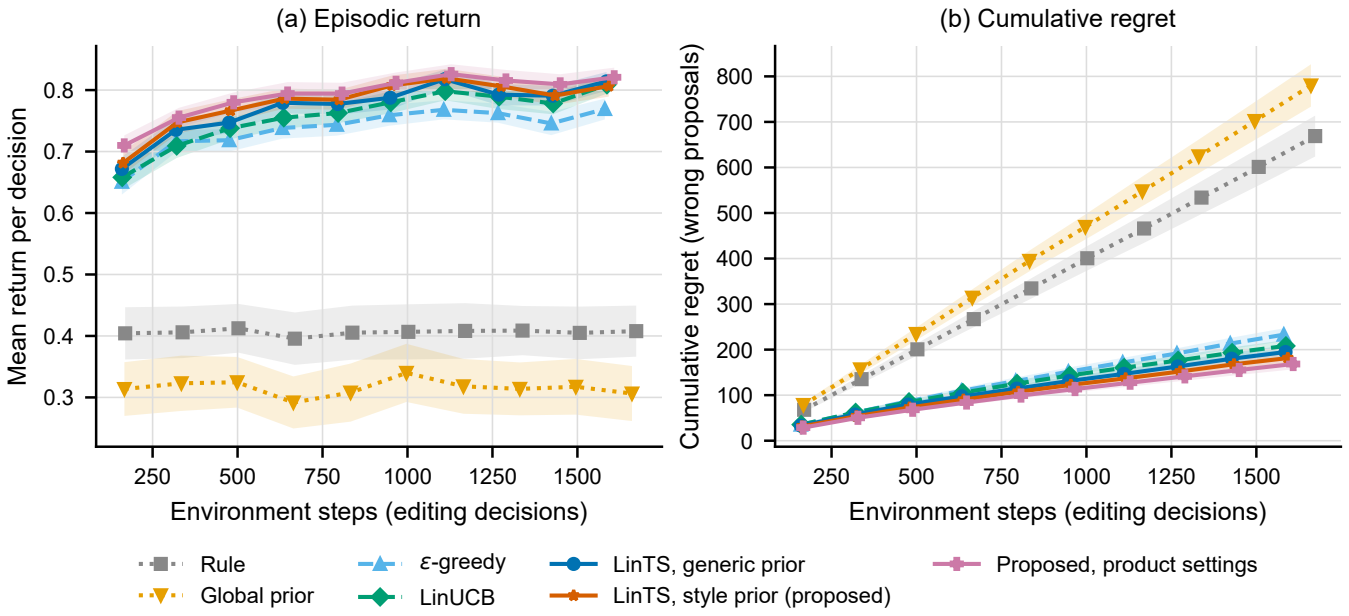


FIGURE 4. (a) Mean return per decision and (b) cumulative regret over environment steps, across six creators and ten seeds. Bands show 95 percent confidence intervals. The learners rise quickly and stabilize, and their bands narrow, whereas the fixed policies stay flat.

TABLE 6. Comparison With Benchmark Methods (Six Creators, Ten Seeds, Ten Videos; ↑ Higher Is Better, ↓ Lower Is Better)

Method	Video 1 ↑	Video 5 ↑	Video 10 ↑	Return ↑	Success ↑	Steps to 85% ↓	Regret ↓	Corr./min ↓
Rule (one-click tool)	0.594	0.596	0.586 ± 0.031	0.408	0.000	never	668.9 ± 45.0	19.91
Global prior, no learning	0.525	0.513	0.521 ± 0.035	0.306	0.000	never	780.1 ± 46.5	22.77
ϵ-greedy	0.777	0.862	0.874 ± 0.011	0.769	0.700	519.0	232.8 ± 14.1	7.64
LinUCB	0.779	0.873	0.903 ± 0.011	0.809	0.823	517.5	208.1 ± 13.0	6.66
Neural, blank	0.735	0.894	0.915 ± 0.011	–	0.880	402.5	186.7 ± 13.6	–
Neural, pretrained	0.789	0.900	0.921 ± 0.011	–	0.947	341.5	167.8 ± 14.2	–
NeuralLinear, blank	0.677	0.880	0.907 ± 0.013	–	0.883	528.0	219.6 ± 15.0	–
NeuralLinear, pretrained	0.724	0.880	0.916 ± 0.012	–	0.857	481.0	202.3 ± 13.6	–
LinTS, generic prior	0.790	0.891	0.904 ± 0.011	0.814	0.853	430.0	194.7 ± 14.7	6.27
LinTS, style prior (proposed)	0.806	0.890	0.910 ± 0.012	0.807	0.903	344.0	181.5 ± 11.0	6.19
Proposed, retuned	**0.826**	**0.909**	**0.931 ± 0.008**	–	**0.967**	**334.0**	**152.7 ± 10.6**	–
Proposed, product settings	0.825	0.900	0.912 ± 0.012	0.821	0.907	343.5	168.4 ± 11.7	5.82

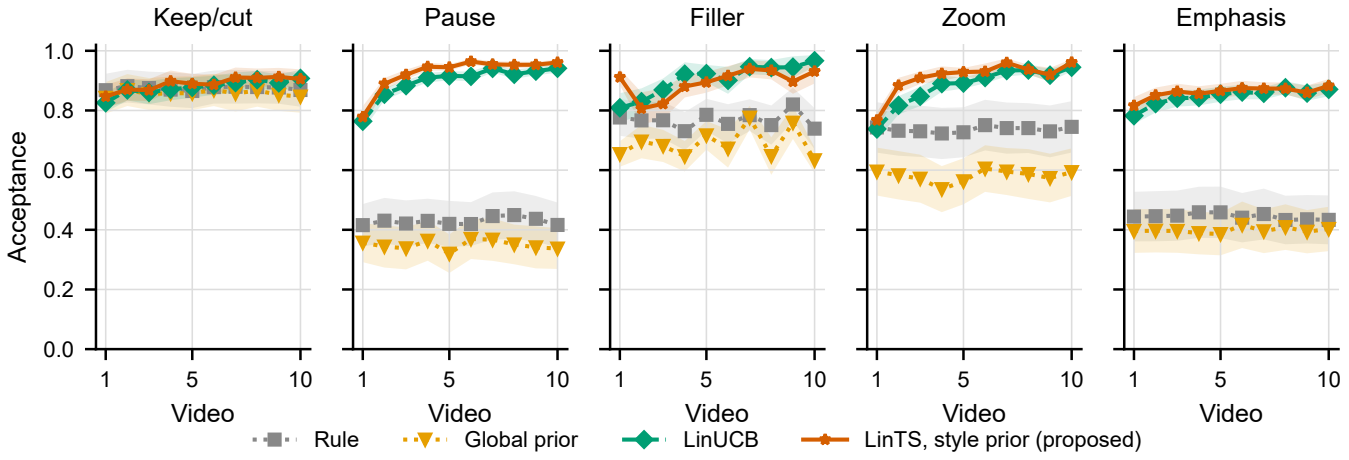


FIGURE 5. Acceptance per decision type over ten videos. The gain from personalization is largest on pauses, which is where creators' preferences differ the most. On keep/cut, a good rule is already close to the learners.

TABLE 7. Acceptance on the Tenth Video per Simulated Creator

Creator	Rule	Global	LinUCB	LinTS	Proposed
Snappy	0.701	0.644	0.881	0.868	**0.893**
Relaxed	0.472	0.348	0.956	0.961	**0.974**
Storyteller	0.671	0.600	**0.912**	0.906	0.910
Minimalist	0.422	0.346	0.860	**0.870**	0.857
Focused	0.531	0.535	**0.933**	0.929	0.915
Blend	0.721	0.652	0.875	0.887	**0.910**

TABLE 8. Acceptance on the Tenth Video per Decision Type

Method	Keep/cut	Pause	Filler	Zoom	Emphasis
Rule	0.869	0.416	0.739	0.745	0.433
Global prior	0.847	0.337	0.633	0.593	0.403
LinUCB	0.907	0.941	0.967	0.945	0.871
LinTS, generic prior	0.904	0.953	0.931	0.963	0.868
Proposed	0.905	0.961	0.931	0.961	0.883
Product settings	0.876	0.983	0.943	0.973	0.889

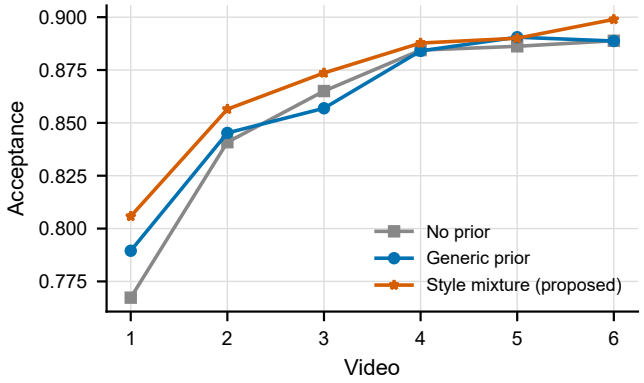


FIGURE 6. Acceptance on the first six videos under three priors, with the evaluated style excluded from the prior. The style mixture leads early and the curves meet as personal data accumulates.

Table 10 splits the result by target. Tighter targets are harder for the bandit, with 54.7 percent error at 20 s, as a 20 s target needs more cutting than any learned habit provides, whereas the planner stays under 6 percent at all three.

5) Does the Queue Save Effort?

Review order matters most when a creator checks few decisions (Fig. 10). At 30 percent, an uncertainty-ordered queue gives 0.869 correct decisions in the export, against 0.784 for random order and 0.633 for timeline order, and all three converge near 0.90 at full review. Confidence predicts mistakes well enough that the same effort fixes more of them. Policies trained at a 30 percent budget also score higher when frozen after uncertainty-ordered review (0.841 against 0.800), a hint that the queue selects useful training samples, though the difference is near the width of the intervals.

6) Can Confidence Be Trusted?

The queue depends on confidence tracking mistakes, which we measured directly (Table 11). At $v = 0.25$ the expected calibration error over about 58 000 suggestions is 0.023. Suggestions below the 0.7 threshold make up 10.1 percent of the total and are right only 37.1 percent of the time, against 94.4 percent above it, with an area under the curve of 0.89 for telling right from wrong. A creator who checks only the flagged tenth is looking where errors concentrate.

A smaller v loses this signal. At $v = 0.05$ acceptance rises (0.916 against 0.902 late in training), but the area under the

TABLE 9. Computational Cost of the Learner (One Thread)

Operation	Mean	95th percentile
Thompson choice with 200 draws (ms)	0.07	0.10
Update of one arm (ms)	0.02	0.03
Plan one video, 65 stages (ms)	186	197
Parameters per creator	305	
Data statistics per creator (KiB)	67	
Simulated run, 10 videos, 1600 decisions (s)	2.4	

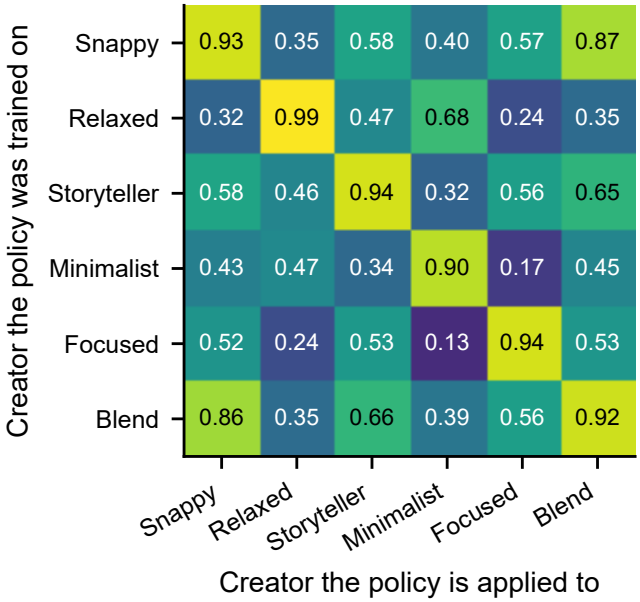


FIGURE 7. Frozen policies applied to each creator on unseen videos. The dominant diagonal shows that learned policies are specific to the creator they were trained on.

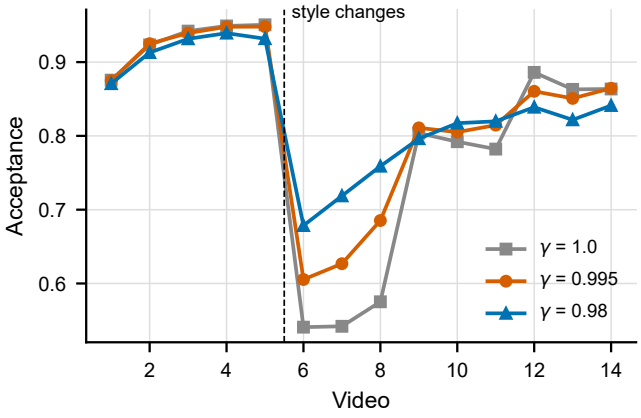


FIGURE 8. Style change at video 6 for three discount factors. Stronger discounting softens the drop at the switch but lowers the level reached before and after it.

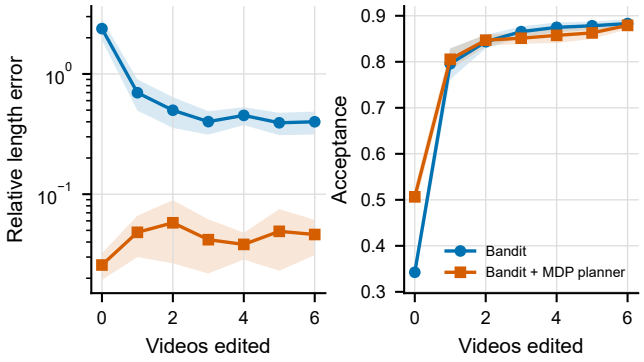


FIGURE 9. Unseen videos for a creator with a target length. Left: relative length error on a logarithmic axis. Right: acceptance.

TABLE 10. Relative Length Error on Unseen Videos by Target

Method	20 s	30 s	45 s
Bandit only	0.547	0.340	0.314
Bandit with planner	0.054	0.052	0.033

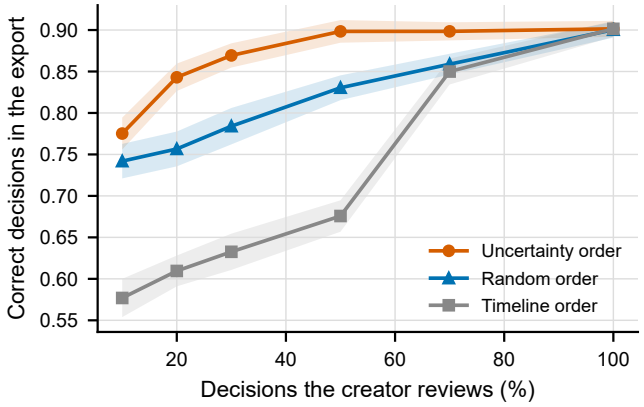


FIGURE 10. Quality of the exported edit against review budget. Uncertainty order dominates below full review and converges with the others at 100 percent.

curve falls to 0.72 and only 3.2 percent of suggestions fall below the threshold, leaving many mistakes unflagged. At $v = 0.5$ the ranking sharpens (0.94), but 18.6 percent are flagged and acceptance falls to 0.876. The experiments use $v = 0.25$ and the product $v = 0.15$, chosen between those extremes: on 120 fresh runs the product's values reached 0.903 against 0.892 after six videos (paired difference 0.011 $\pm$ 0.005). Under our feedback model, then, Thompson sampling does not seem to help learning. Its contribution is an uncertainty signal a creator can use.

7) Can a Policy Be Judged Offline?

Fig. 11 and Table 12 evaluate policies from LinTS logs, with simulator truth as reference. On average the doubly robust estimate ranks the learned policy first by a wide margin and comes within 1.3 points of it (true 0.932, DR 0.945), and it recovers the order of Rule (0.749, DR 0.739) and the generic prior (0.703, DR 0.694). DM has the lowest error

TABLE 11. Confidence as a Predictor of Mistakes

Measure	$v = 0.05$	$v = 0.25$	$v = 0.5$
Expected calibration error	0.065	0.023	0.023
Area under the curve (right vs. wrong)	0.715	0.893	0.940
Share flagged (confidence < 0.7)	0.032	0.101	0.186
Correct among flagged	0.296	0.371	0.372
Correct among not flagged	0.919	0.944	0.963
Late acceptance (videos 8–10)	0.916	0.902	0.876

TABLE 12. Off-Policy Estimators Against Simulator Truth

Estimator	Bias	RMSE	95% CI coverage	Picks best
DM	0.040	0.065	0.48	1.00
IPS	−0.007	0.153	0.72	0.72
SNIPS	0.013	0.124	0.81	0.97
DR	0.002	0.126	0.91	0.94
SNDR	0.003	0.121	0.86	0.94

(RMSE 0.065) but its bias makes its intervals almost useless, covering the truth only 48 percent of the time. DR has much smaller bias (0.002) and the best coverage, 91 percent. The small-sample t correction raised coverage from the 71 to 78 percent we saw with percentile intervals, but it still falls short of the nominal 95 percent. Part of the shortfall is likely adaptive logging [53]. The effective sample size is just 15 percent of the 950 logged decisions per creator, because LinTS quickly becomes confident and stops supporting other actions. Offline evaluation in this setting therefore works for ranking clearly different policies, but not for detecting small differences.

8) Real Audio

Speech analysis cannot be judged in simulation. We therefore ran three clips through the actual pipeline and compared a new creator's first cut with a reference edit (Table 13). The first, 44.2 s long, has four abandoned attempts at one sentence before the clean take and unrelated background speech in a 7 s gap. The first cut drops every abandoned attempt. The second runs 238.5 s, with long gaps, six retaken lines, and five phrases spoken to someone off camera. All five were cut, and the call to action at the end was kept. The third is a 54.5 s scripted stress clip with a synthetic voice, containing fillers, a stutter, hedges, a three-attempt restart, and a sentence whose first delivery is the good one. The agent resolved all of these and retained one filler ("basically") with no audio dip around it.

These are encouraging results but very thin evidence. The sample is three clips and one of them has a synthetic voice. The references were prepared with AI assistance from the transcript and from decoding doubtful stretches on their own, not by an independent editor listening, and they use the transcript's wording, so zero differences does not mean perfect transcription: the system writes "Klein" for "client", and no reference corrects it. The crew rules and the process isolation were developed on these same clips, so they are not held out. Through this comparison we found and fixed a stuttered scrap of a restart and the lost sentence described

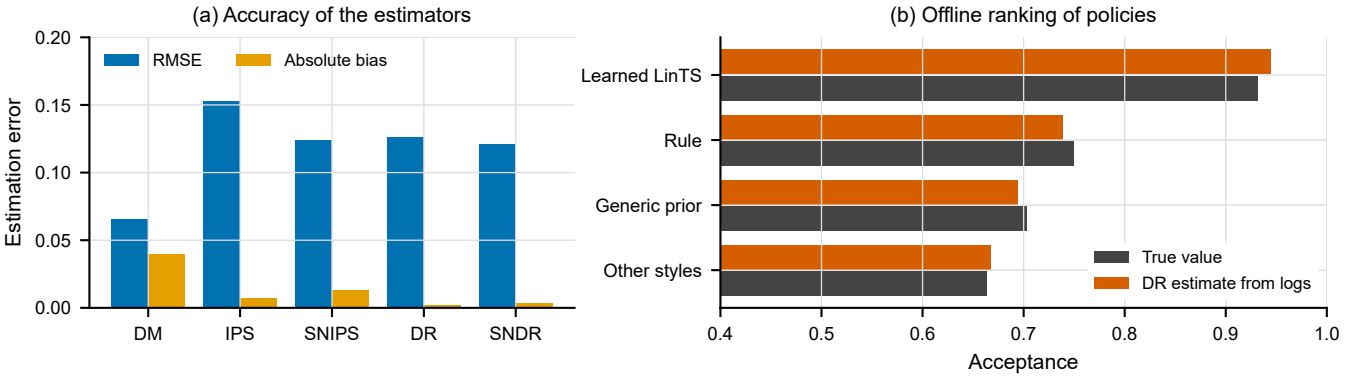


FIGURE 11. Off-policy evaluation against simulator truth. (a) Error of each estimator. (b) True value and doubly robust estimate of four target policies, computed only from the logs of the learning agent.

TABLE 13. First Cut Against a Reference Edit on Recorded Clips

Measure	Clip A	Clip B	Stress clip
Source length (s)	44.2	238.5	54.5
First cut length (s)	22.1	72.6	19.2
Clips in the edit	6	24	12
Suggestions / confidence under 0.7	56 / 11	182 / 17	80 / 6
Retake phrases cut	4	6	5
Crew phrases cut	0	5	0
Reference words	93	220	70
Word differences from reference	0	0	1

Differences are measured on caption words. References keep the transcript's wording, so this checks which lines survive, not transcription accuracy.

earlier. There are certainly more such problems in footage we have not tried. We also ran the editor ourselves on these recordings during development and turned what we saw into fixes; this feedback was informal and is not a user study.

9) What Did the Model Learn?

To see whether the learned weights make sense, we trained with no prior for ten videos and six seeds, so that everything comes from feedback, and scored hand-built contexts with the posterior means (Table 14). An entry's sign shows the favored action, and the results read like the personas' hidden rules. All three creators remove "um"; only Snappy removes "like". Snappy removes a 0.6 s pause and Relaxed firmly keeps it, then nearly flips between 0.6 s and 1.5 s, the threshold in its rule. Every style cuts an earlier take and keeps an ordinary phrase. These weights feed the profile page, and with a linear model the check takes a few lines of code.

D. COMPARISON WITH EXISTING METHODS

Table 6 is the main benchmark. Against the Rule baseline, modeled on one-click tools, the proposed method cuts regret by 73 percent and corrections per minute of footage by 69 percent, from 19.9 to 6.2. The unadapted Global prior does even worse than Rule, because a pooled model suffers when styles disagree. Differences among learners are much

TABLE 14. Learned Advantage of One Action Over Another (Posterior Means, No Prior)

Context and contrast	Snappy	Relaxed	Storyteller
"um": REMOVE − KEEP	+0.94	+0.95	+0.49
"like": REMOVE − KEEP	+0.91	−0.49	−0.89
Pause 0.6 s: REMOVE − KEEP	+0.88	−0.78	+0.38
Pause 1.5 s: REMOVE − KEEP	+1.09	+0.07	+0.48
Earlier take: CUT − KEEP	+0.63	+0.39	+0.60
Ordinary phrase: CUT − KEEP	−0.69	−1.01	−0.73

smaller. Video 10 acceptance for LinUCB (0.903) and proposed (0.910) overlaps within intervals. On matched persona and seed, the proposed method's regret advantage is 26.6 $\pm$ 9.1 decisions over LinUCB and 13.3 $\pm$ 9.3 over plain LinTS, both with $p < 0.01$, while on video 10 alone it is not distinguishable from either ($p = 0.15$ and $p = 0.22$). No correction for multiple comparisons was applied.

1) A Fair Test Against Neural Agents

Our first deep baseline made the linear model look better than it deserved. That search tried four settings, and its best ϵ was the smallest on offer, a sign that the grid was too narrow. We repeated the comparison with one procedure for every family: settings chosen on seeds 201–203 by acceptance on videos 5 and 6, and results reported on the usual seeds, with each run extended to 50 videos. The neural agent keeps one network per decision type, trained online from a replay buffer, which is the DQN recipe [2] with $\gamma = 0$ because these decisions have no transition. Its grid now has 36 settings of learning rate, ϵ, gradient steps per event, and width. We added NeuralLinear [29], which fits a Bayesian linear model on the last hidden layer and acts by Thompson sampling. Pretrained variants learn first from the other creators' labels, the same information the generic prior has. Our own method went through 60 settings of λ, v, and ϵ.

This changed our conclusion (Fig. 12, Table 15). With wider tuning, the networks overtake our default settings: pretrained Neural makes 13.7 $\pm$ 6.9 fewer wrong proposals over ten videos and 94.5 $\pm$ 19.4 fewer over fifty. Retuning (Table 5) puts the linear method back in front, with 15.0 $\pm$

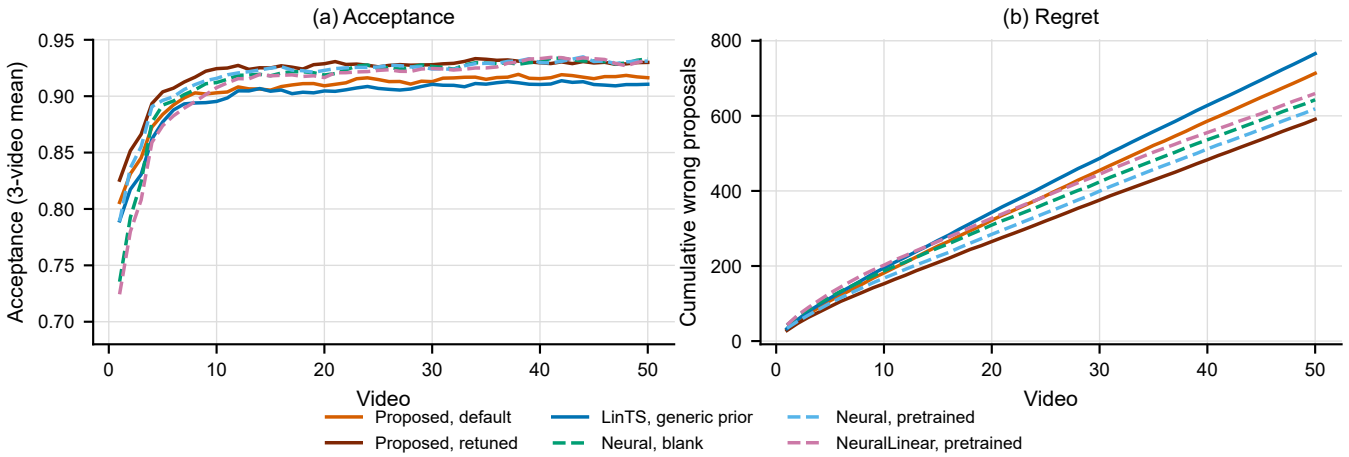


FIGURE 12. Comparison over 50 videos with every family tuned on the same separate seeds. The retuned method leads early and keeps the lowest cumulative regret, while network-based agents draw level on acceptance.

TABLE 15. Fifty Videos, Six Creators, Ten Seeds

Method	Video 50	Regret, 10	Regret, 50
LinTS, generic prior	0.910	194.7	765.7
Proposed, default	0.915	181.5	713.3
Proposed, retuned	0.927	**152.7**	**590.9**
Neural, blank	0.929	186.7	642.9
Neural, pretrained	0.928	167.8	618.8
NeuralLinear, blank	**0.933**	219.6	668.9
NeuralLinear, pretrained	**0.933**	202.3	659.5

TABLE 16. Visual Rhythm on Unseen Videos (Four Creators, 20 Seeds)

Agent	Match, video 3	Match, video 8	Rating, video 8
Fixed rule	0.471	0.471 ± 0.036	-0.42 ± 0.13
Linear bandit	0.843	0.919 ± 0.015	$+0.04 \pm 0.20$
RLSVI	0.848	0.901 ± 0.020	$+0.04 \pm 0.19$
DQN, blank	0.815	0.855 ± 0.026	$+0.04 \pm 0.19$
DQN, warm start	0.805	0.865 ± 0.023	$+0.03 \pm 0.19$
PPO, blank	0.311	0.532 ± 0.048	-0.25 ± 0.19
PPO, warm start	0.514	0.542 ± 0.053	-0.11 ± 0.17

6.7 fewer wrong proposals than pretrained Neural over ten videos ($p < 0.001$), 27.9 ± 19.7 fewer over fifty ($p = 0.006$), and a higher start, 0.826 against 0.789. On 120 fresh runs (seeds 301–320), retuned settings beat the defaults by 1.5 ± 0.5 points at video 10 and by 30.8 ± 3.6 wrong proposals. By video 50 the retuned method is statistically level with Neural (0.927 against 0.929) and slightly below NeuralLinear (0.933, $p = 0.09$).

Retuning costs confidence quality, which acceptance figures hide. Calibration error rises from 0.023 to 0.052, the area under the curve falls from 0.893 to 0.831, and only 6.0 percent of suggestions fall below the review threshold instead of 10.1. Anyone relying on the review queue may prefer the default, so most results in this paper use it. Two cautions remain. The chosen neural learning rate was the lowest in its grid, and a lower one might do better. All of these creators can also be described by our features, which helps a linear model. We therefore defend the linear method on early mistakes, cost, exact replay, and readable explanations, and we make no claim that networks are worse. With enough feedback, a network is a reasonable choice.

2) Visual Rhythm: Is a Sequential Agent Needed?

Table 16 and Fig. 13 test the sequential agent on four simulated creators whose preferences depend on history: one modest punch-in after a quiet stretch, a strong beat only on important phrases after eight quiet seconds, a beat every three seconds without repeating the kind, and almost nothing. Each edits eight videos with feedback on every beat and rates each whole video by how close its beats per minute are to what the creator likes. Scores are on four unseen videos, over 20 seeds not used for tuning. DQN and PPO were tuned over learning rate and ϵ or update epochs on separate seeds, from a blank start and from a start fitted to the same rule that gives RLSVI its prior.

RLSVI did not beat the bandit. Its final match is 0.901 against 0.919 (paired difference -0.018 ± 0.019, $p = 0.055$), and their ratings are level at +0.04. We expected the sequential agent to win, and the likely reason it did not lies in our design. Every state these creators react to is also visible to the bandit through the history features, so choosing the best action for the current state is already close to optimal. The delayed rating adds little that per-beat feedback has not already said. The largest difference is on the minimal creator (0.816 against 0.894), where the agent chases rare positive beats. Both agents still fail on rating for the two creators who want few beats, because a few wrong beats are enough to miss a narrow target rate. The deep agents had far less to work with, about 260 samples per run. The better DQN reached 0.865 and the better PPO 0.542, so at this data budget both trail the linear agents on matches even after tuning, although DQN's final rating is level with theirs and every interval on rating is wide. These creators give feedback on every beat, which leaves little for a delayed reward to add.

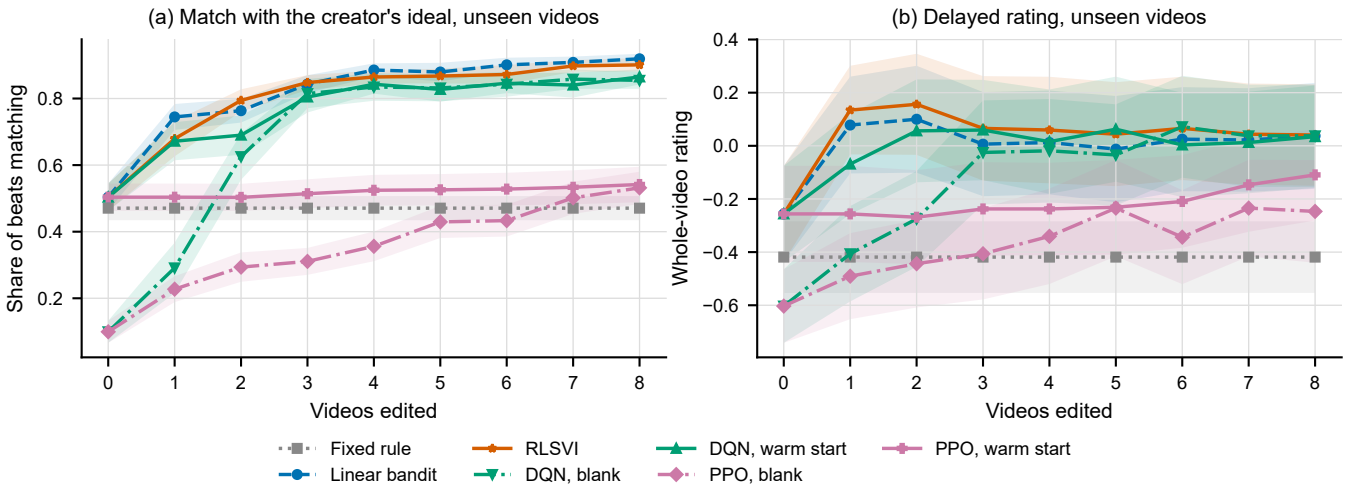


FIGURE 13. Visual rhythm on unseen videos. (a) Share of beats that match the creator's ideal. (b) Whole-video rating, in $[-1, 1]$. Bands show 95 percent intervals over 80 runs per agent.

TABLE 17. Visual Rhythm With Sparse Feedback (Video 16, Match / Rating)

Beats with feedback	RLSVI	Bandit	DQN, warm
100%	0.915 / +0.06	0.945 / +0.03	0.906 / +0.00
30%	0.825 / +0.18	0.910 / +0.06	0.808 / +0.04
10%	0.687 / +0.24	0.828 / +0.04	0.670 / +0.01
Rating only	0.336 / +0.16	0.515 / −0.32	0.276 / −0.31

To give RLSVI its fair test, we reran the agents for 16 videos while withholding per-beat feedback, down to the whole-video rating alone (Table 17). Here the picture splits. RLSVI's rating overtakes the bandit's as feedback thins, by 0.21 ± 0.11 at 10 percent and by 0.48 ± 0.15 at rating only (both $p < 0.001$), while its beat match falls well behind (0.336 against 0.515). In other words, it learns how often a creator wants beats without learning where to put them. Part of this rating gap is built in, because a bandit in this setup cannot use the rating at all. DQN receives the rating too and does not benefit (rating −0.31). Because all settings were tuned under full feedback, these figures may understate both deep and randomized agents.

E. ABLATION STUDY

We removed one component at a time, with paired seeds (Table 18, Fig. 14). Manual-edit mapping adds 2.5 points on early videos and 1.1 late, since half of the corrections would otherwise be lost. Retake features add 2.1 early and 1.6 late, the largest lasting effect, as every persona cuts earlier takes. The style prior adds 1.5 early and 0.8 late. Removing the prior altogether gives the largest early drop among prior variants (2.1) with no significant late penalty. Semantic and history features have almost no effect in the simulator, which for semantic features may partly reflect synthetic embeddings being crude stand-ins for language.

Three removals do not hurt, and two improve the score. Removing implicit feedback changes little (+0.2 late, within noise). Removing discounting adds 1.8 points late, because these personas do not drift; Fig. 8 shows the opposite case. Exploration is the surprise: a policy with $v = 0$ and $\epsilon = 0$ scores 2.7 points higher early and 2.6 later, with regret of 140.8 against 181.5.

Acceptance during training counts every exploratory proposal as a mistake and cannot say whether exploration led to a better policy. We therefore trained each variant on ten videos under three feedback conditions, froze its posterior mean, and

TABLE 18. Ablation: Change in Acceptance Against the Full Method

Removed	Videos 1–3	Videos 8–10	Regret
Nothing (full method)	0.845	0.903	181.5
Manual-edit mapping	−0.025	−0.011	206.5
Prior altogether	−0.021	−0.003	198.2
Retake features	−0.021	−0.016	213.1
Style prior (generic kept)	−0.015	−0.008	194.7
Semantic features	−0.003	−0.003	190.3
Implicit feedback	+0.004	+0.002	177.9
History features	+0.005	−0.005	186.2
Discounting	+0.003	+0.018	168.5
Exploration	+0.027	+0.026	140.8

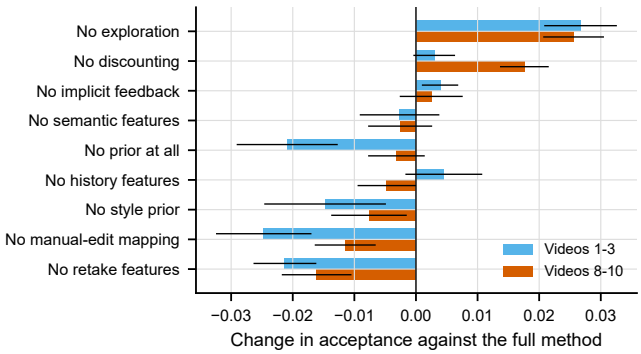


FIGURE 14. Paired change in acceptance from removing each component. Negative bars mean that the component helps. Error bars are 95 percent intervals.

TABLE 19. Exploration Variants: Acceptance During Training and Frozen on Unseen Videos

	Normal		Reluctant		Review 30%	
Variant	Train	Held	Train	Held	Train	Held
Greedy	0.908	0.931	0.884	0.917	0.867	0.908
Thompson, $v = 0.05$	0.897	0.934	0.873	0.914	0.852	0.902
Thompson, $v = 0.25$	0.883	0.932	0.855	0.921	0.837	0.900
Thompson, $v = 0.5$	0.851	0.933	0.810	0.918	0.800	0.906
LinUCB	0.880	0.936	0.841	0.921	0.825	0.911
ϵ-greedy, $\epsilon = 0.1$	0.860	0.932	0.832	0.913	0.816	0.903

Reluctant: correction probability 0.2. Held-out intervals are about ± 0.01 throughout.

scored it on four unseen videos with no corrections (Table 19, 48 runs per cell). Held-out acceptance barely depends on the exploration strategy: every paired difference from Thompson sampling at $v = 0.25$ lies within about one point, and most intervals include zero. A purely greedy learner ends at 0.931 against 0.932 under normal feedback. Training cost is large and visible to the creator, with 0.908 for greedy against 0.883 for $v = 0.25$ and 0.851 for $v = 0.5$.

The usual argument for exploration fails here for three reasons. Each video brings fresh contexts, and even a greedy agent sees many feature combinations. A creator's override names the correct arm and updates both arms, which comes close to full information. A prior mean also gives an informed start. The simulator has no rejections that withhold the right answer, where exploration should matter most, so we claim only that exploration has not helped under the feedback we modeled. We keep Thompson sampling in the deployed system for practical reasons: it supplies the confidence that orders the review queue and the propensities that off-policy evaluation needs, and a greedy logger would leave nothing to reweight. A smaller v scores higher on acceptance, at the price shown in Table 11.

F. SENSITIVITY ANALYSIS

We varied one hyperparameter at a time over six seeds per persona (Fig. 15). Prior strength λ has a broad optimum near 2 to 10 late in training. Weak priors ($\lambda = 0.5$) lose the cold-start benefit, at 0.818 early, and very strong ones ($\lambda = 30$) learn too slowly (0.807 early, 0.884 late), a loss the product accepts for keep/cut alone to prevent one click from causing large changes. Posterior scale v has the largest effect: at $v = 2$ the agent is little better than a rule (0.697 late), and it improves monotonically down to the smallest value tried ($v = 0.05$, 0.916 late). The exploration floor ϵ behaves similarly, from 0.913 at 0 to 0.819 at 0.2. Discount γ gives 0.918 late at 1.0 and 0.858 at 0.95. Implicit reward has a small effect, with lower values marginally better (0.909 at 0, 0.893 at 1).

Late acceptance intervals are about ± 0.012 throughout, so we do not read differences below two points between settings as meaningful. Across all five panels, less randomness and less forgetting improve stationary acceptance, the same message as the ablation.

TABLE 20. Creators Outside the Model Family, 30 Videos (Acceptance / Wrong Proposals)

Creator	LinTS	Neural	Neural, pretr.	Fixed
Nonlinear	0.931 / 454	0.948 / 244	0.953 / 230	0.479
Noisy	0.953 / 541	0.880 / 564	0.873 / 532	0.836
Arbitrary	0.508 / 1159	0.511 / 1140	0.477 / 1142	0.559

Acceptance counts keep/cut, pause, and filler decisions; wrong proposals count all decisions.

G. ROBUSTNESS AND GENERALIZATION

We stressed the environment in three ways (Fig. 16). Raising label noise from 0 to 30 percent lowers acceptance on videos 8 to 10 steadily from 0.920 to 0.749, still well above Rule (about 0.59). Reviewing just 10 percent of decisions still reaches 0.825, and 30 percent reaches 0.873, against 0.902 at full review. Reluctance to correct matters little: dropping the correction probability from 1.0 to 0.2 lowers acceptance only from 0.910 to 0.889, because accepts still carry information. Video length also has little effect. With 12, 30, and 60 phrases per video (about 67, 160, and 315 decisions), acceptance is 0.898, 0.902, and 0.912, consistent with learning being measured in decisions rather than videos.

To test conditions the model and its prior were not built for, we placed creators outside both the model family and the prior's training data, scoring only keep/cut, pause, and filler decisions since these personas never zoom. By video 8 (Fig. 17), LinTS reaches 0.883 with nonlinear rules, well above Rule (0.511) and the best fixed choice per type (0.489), because binned features can approximate interval rules. With noisy feedback (20 percent wrong labels, 30 percent correction rate) it reaches 0.810, still below the best fixed arm at 0.835. With arbitrary rules that depend on a hash of the unit, it ends at 0.499, below the fixed arm at 0.568, because nothing can be learned there and the agent chases noise.

Table 20 extends these creators to 30 videos and adds the tuned networks, with eight seeds per cell, so intervals are wide. Each family wins where its assumptions hold. On nonlinear rules the network makes roughly half as many mistakes as LinTS. Under noise the order reverses: LinTS passes the fixed arm and beats blank Neural by 7.3 ± 3.8 points. On arbitrary rules every learner stays below a constant policy, so the method is not winning by construction.

H. REPRODUCIBILITY

Every simulated result can be regenerated from the repository, which contains the simulator, the personas, and the experiment scripts behind each table and figure. A complete rerun of the comparison, ablation, sensitivity, and robustness groups involves about 2500 simulated creators and takes under half an hour on the laptop described above, and the final deep comparison takes about an hour and a half. Seeds are fixed per run. Synthetic videos are deterministic functions of their seed, and all methods compared at a given seed see the same videos, which is what makes the paired comparisons valid. Priors are fitted on separate videos and with the evaluated persona excluded. The real clips cannot

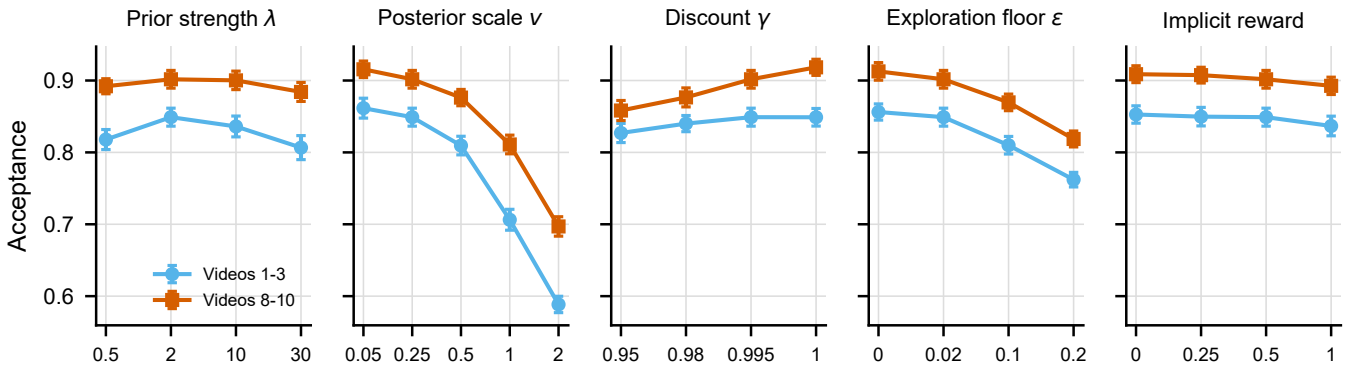


FIGURE 15. One-at-a-time sensitivity of acceptance on early and late videos. Posterior scale and exploration floor have the largest effects; the others are comparatively flat near their chosen values.

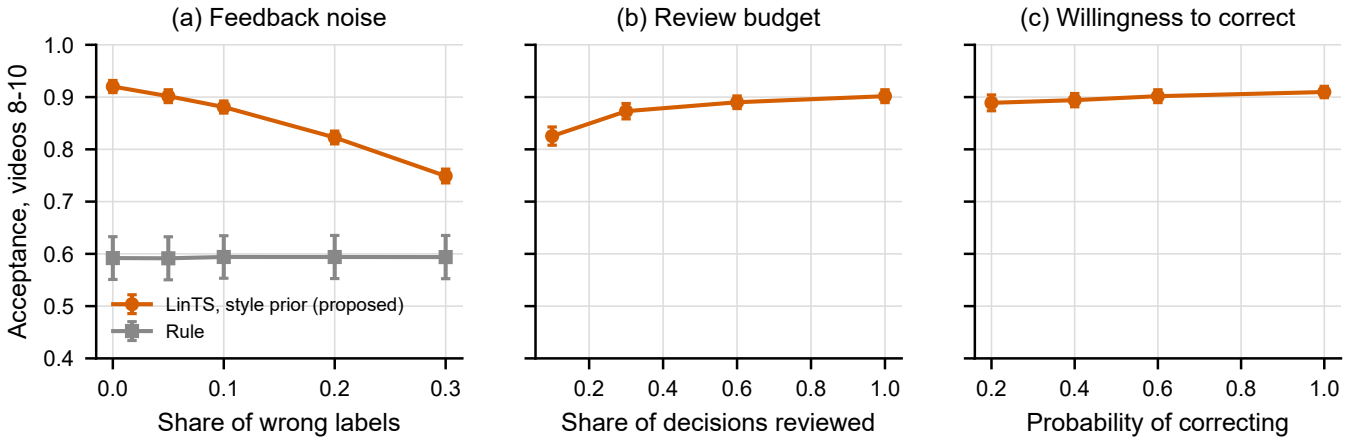


FIGURE 16. Acceptance on videos 8 to 10 under (a) wrong feedback labels, (b) smaller review budgets, and (c) reluctance to correct. Degradation is gradual in all three cases.

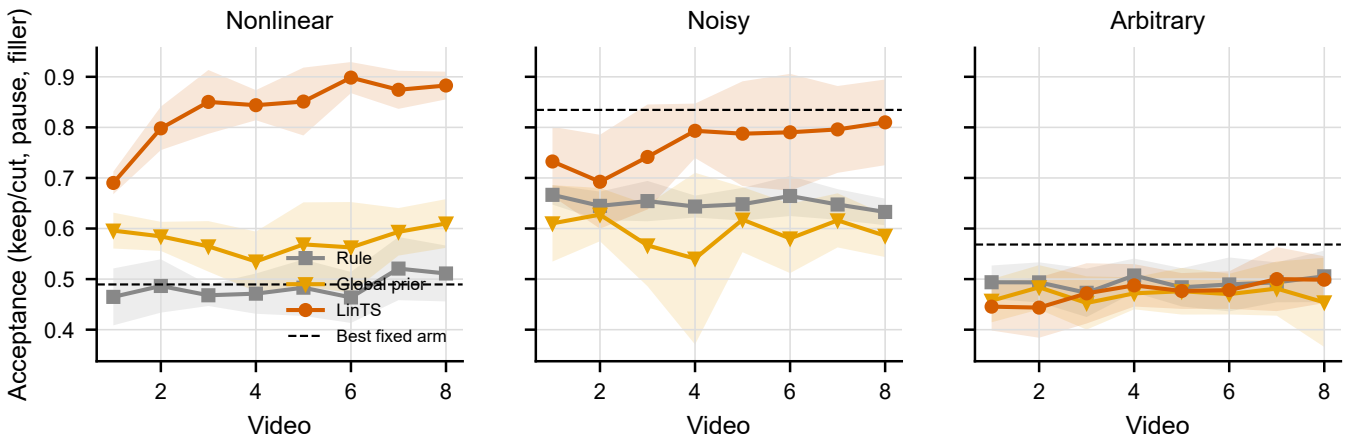


FIGURE 17. Creators outside the model family and the prior's training data. The first eight videos. The method generalizes to nonlinear preferences but is still behind a constant policy under heavy noise and arbitrary preferences.

be redistributed without the rights holder's consent, so the real-audio results can be repeated only with other footage. Repeating the analysis three times on the two real clips in fresh processes gave identical transcripts and cuts. All reported intervals are 95 percent t intervals across runs, and we state explicitly where a comparison is paired.

I. HOW MUCH OF THIS IS REINFORCEMENT LEARNING?

Is this RL, or online regression with a user interface? The agent sees feedback only on what it proposed, its data depend on its own past actions, and it acts by sampling from an explicit posterior. It learns per user under changing preferences, decides what to ask, logs propensities for counterfactual evaluation, and plans over an episode with a learned reward. In the rhythm agent, its actions change the state it acts in next. A contextual bandit is the one-state special case of RL [1], [32], and our planner is an MDP solved by posterior sampling.

The claim has limits too. On binary keep/cut and filler decisions a rejection reveals the right label, which is close to online supervised learning. Exploration did not improve held-out policies. Most actions affect later states only weakly, through history features that also proved to have little effect, and where the state does matter, a sequential agent beat a bandit only on whole-video rating under scarce feedback. We call the method a contextual bandit system with a planning layer, not deep or fully sequential RL. That matches the data we have: a few hundred human signals would not justify a larger model.

J. THREATS TO VALIDITY

Internal validity. The experiments share a code base with the product, and that code was changing while they ran. We reran the simulator experiments in one session for this paper, and repository scripts regenerate every table and figure, but the analysis code was still being revised for real audio. Hyperparameters were first tuned on the same simulator used for evaluation, which favors the method. Later results separate the two: rhythm settings, the deep baselines, and the retuned linear settings were tuned on seeds not used for reporting, and the retuned and product settings were confirmed on fresh seeds. The default $v = 0.25$ was picked for the quality of its confidences, so it does not inflate the headline figure. Earlier tuning may still have affected other results.

Construct validity. Acceptance may be the wrong target. A creator might accept a mediocre suggestion because correcting takes effort. Our reward separates an explicit accept from an untouched suggestion for this reason, but it cannot see what a person would have preferred after more thought. Matching a creator's past decisions also differs from making a video audiences enjoy, and we have no audience data.

External validity. This is the largest threat. Synthetic transcripts contain the mistakes we thought to plant, in proportions we chose. Their semantic embeddings are random vectors around a topic, so the near-zero effect of semantic features in the ablation may not carry over to real speech. The arbitrary creators show the method can lose when its assumptions fail, and real people will fall somewhere between our personas and that extreme.

IV. LIMITATIONS

No formal human study has been run, and this is the most serious limitation. Every learning result concerns simulated creators and features that we designed, and most persona rules can be expressed in those features. The creators outside the model family push against this, but acceptance numbers are not predictions for people, who may be less consistent, tire of reviewing, or hold preferences our features cannot express. The real test of the product claim, time-to-post against a simple editor, is still ahead. Our database also holds too few real logs for off-policy conclusions.

Only three clips have checked the analysis, two real and one with a synthetic voice, against references prepared from the transcripts, using word-level comparison and no formal listening test. We wrote the crew rules while looking at one of those clips. Only English is supported; other languages get a warning, not a usable edit. Running Whisper in a fresh process removes a difference we could reproduce but not explain, so it is a workaround. Fillers with no audio dip stay in the edit, disfluency recovery can miss repeats the CTC model does not read clearly, and Whisper can still mishear words such as names.

Binary keep/cut and filler decisions are close to online supervised learning, because a correction reveals the right label. The bandit layer treats decisions as nearly independent, and frozen history features remove most of the sequential interaction from the planner's reward, leaving a length budget. The planner does not improve which phrases are chosen. The rhythm agent has been tested only against simulated creators whose preferences a bandit can also see. Off-policy estimates suffer from an effective sample size of 15 percent and intervals that under-cover, and because the product removes the exploration floor on keep/cut, actions Thompson sampling never picks have no support in real logs.

The learner scales easily, but analysis takes one to two times the clip's length on a laptop CPU, and uploads are limited to five minutes. A wrong CUT on an autopilot export silently deletes content. Autopilot shows a badge based on expected corrections and a trust score, and we have not validated that badge.

V. CONCLUSION AND FUTURE WORK

RoughCut learns one creator's editing style from normal editing, where rule-based tools cannot adapt and supervised models lack data. It pairs a linear Thompson sampling bandit with a tempered style posterior, an uncertainty-ordered review queue, an exact planner for length targets, a value-iteration agent for pacing, and propensity logging. In simulation, retuned settings reach 93.1 percent acceptance by the tenth video against 58.6 percent for a rule baseline and make the fewest wrong proposals over 50 videos of any agent we tried. Policies are strongly creator-specific (0.936 against 0.468 across creators). Review at a 30 percent budget gives 86.9 percent correct decisions against 63.3 percent in timeline order, and the planner cuts length error from 40.0 to 4.6 percent for 0.4 points of acceptance.

Three results limit these claims. Tuned networks overtook our default settings, drew level with the retuned ones on acceptance by the fiftieth video, and did better on nonlinear taste, so the linear choice rests on scarce-feedback performance, cost, and transparency. Retuning for fewer mistakes weakened the confidence signal the review queue needs. And value iteration improved on a bandit only for whole-video rating under scarce feedback, while random exploration showed no value because corrections already carry so much information. Nothing is yet established about real creators.

The next step is a study with real creators, comparing time-to-post and blind quality ratings against a simple editor, with off-policy evaluation on those logs. More real clips need independent reference edits and listening tests. Keep/cut selection under a length target needs a better reward, possibly from pairwise preferences [7]. History features could return to the planner with uncertainty penalties [45], now that we know how they fail, and RLSVI should be tuned for sparse feedback so its rating gain can be tested with people. Crew-chatter detection needs a labeled set and a learned model. Exploration should be kept for where it helps, such as multi-arm decisions with rejection-only feedback. Style prototypes could eventually come from real users' posteriors, as in meta-learning [34], [35].

ACKNOWLEDGMENT

The authors thank Prof. Geetha M for her guidance. Claude, an AI model from Anthropic, was used to assist with software development, experiments, and drafting and editing this manuscript. The authors reviewed the content and take responsibility for it.

REFERENCES

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.

[3] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in Proc. 19th Int. Conf. World Wide Web, Raleigh, NC, USA, 2010, pp. 661–670, doi: 10.1145/1772690.1772758.

[4] G. Zheng et al., "DRN: A deep reinforcement learning framework for news recommendation," in Proc. World Wide Web Conf., Lyon, France, 2018, pp. 167–176, doi: 10.1145/3178876.3185994.

[5] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. H. Chi, "Top-K off-policy correction for a REINFORCE recommender system," in Proc. 12th ACM Int. Conf. Web Search Data Mining, Melbourne, VIC, Australia, 2019, pp. 456–464, doi: 10.1145/3289600.3290999.

[6] Y. Hu, H. He, C. Xu, B. Wang, and S. Lin, "Exposure: A white-box photo post-processing framework," ACM Trans. Graph., vol. 37, no. 2, pp. 1–17, Apr. 2018, doi: 10.1145/3181974.

[7] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in Proc. Adv. Neural Inf. Process. Syst., vol. 30, 2017, pp. 4299–4307.

[8] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Proc. Adv. Neural Inf. Process. Syst., vol. 35, 2022, pp. 27730–27744.

[9] F. Berthouzoz, W. Li, and M. Agrawala, "Tools for placing cuts and transitions in interview video," ACM Trans. Graph., vol. 31, no. 4, pp. 1–8, Jul. 2012, doi: 10.1145/2185520.2185563.

[10] S. Rubin, F. Berthouzoz, G. J. Mysore, W. Li, and M. Agrawala, "Content-based tools for editing audio stories," in Proc. 26th ACM Symp. User Interface Softw. Technol., St. Andrews, U.K., 2013, pp. 113–122, doi: 10.1145/2501988.2501993.

[11] B. Huber, H. V. Shin, B. Russell, O. Wang, and G. J. Mysore, "B-Script: Transcript-based B-roll video editing with recommendations," in Proc. CHI Conf. Hum. Factors Comput. Syst., Glasgow, U.K., 2019, pp. 1–11, doi: 10.1145/3290605.3300311.

[12] W. Chu, L. Li, L. Reyzin, and R. E. Schapire, "Contextual bandits with linear payoff functions," in Proc. 14th Int. Conf. Artif. Intell. Statist., Fort Lauderdale, FL, USA, 2011, pp. 208–214.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, arXiv:1707.06347.

[14] H. V. Shin, W. Li, and F. Durand, "Dynamic authoring of audio with linked scripts," in Proc. 29th Annu. Symp. User Interface Softw. Technol., Tokyo, Japan, 2016, pp. 509–516, doi: 10.1145/2984511.2984561.

[15] A. Truong, F. Berthouzoz, W. Li, and M. Agrawala, "QuickCut: An interactive tool for editing narrated video," in Proc. 29th Annu. Symp. User Interface Softw. Technol., Tokyo, Japan, 2016, pp. 497–507, doi: 10.1145/2984511.2984569.

[16] A. Pavel, C. Reed, B. Hartmann, and M. Agrawala, "Video digests: A browsable, skimmable format for informational lecture videos," in Proc. 27th Annu. ACM Symp. User Interface Softw. Technol., Honolulu, HI, USA, 2014, pp. 573–582, doi: 10.1145/2642918.2647400.

[17] M. Leake, A. Davis, A. Truong, and M. Agrawala, "Computational video editing for dialogue-driven scenes," ACM Trans. Graph., vol. 36, no. 4, pp. 1–14, Jul. 2017, doi: 10.1145/3072959.3073653.

[18] O. Fried et al., "Text-based editing of talking-head video," ACM Trans. Graph., vol. 38, no. 4, pp. 1–14, Jul. 2019, doi: 10.1145/3306346.3323028.

[19] B. Wang, Y. Li, Z. Lv, H. Xia, Y. Xu, and R. Sodhi, "LAVE: LLM-powered agent assistance and language augmentation for video editing," in Proc. 29th Int. Conf. Intell. User Interfaces, Greenville, SC, USA, 2024, pp. 699–714, doi: 10.1145/3640543.3645143.

[20] M. Huh, D. Li, K. Pimmel, H. V. Shin, A. Pavel, and M. Dontcheva, "VideoDiff: Human-AI video co-creation with alternatives," in Proc. CHI Conf. Hum. Factors Comput. Syst., Yokohama, Japan, 2025, pp. 1–19, doi: 10.1145/3706598.3713417.

[21] M. Wang, G.-W. Yang, S.-M. Hu, S.-T. Yau, and A. Shamir, "Write-A-Video: Computational video montage from themed text," ACM Trans. Graph., vol. 38, no. 6, pp. 1–13, Nov. 2019, doi: 10.1145/3355089.3356520.

[22] K. Zhou, Y. Qiao, and T. Xiang, "Deep reinforcement learning for unsupervised video summarization with diversity-representativeness reward," in Proc. AAAI Conf. Artif. Intell., vol. 32, no. 1, 2018, doi: 10.1609/aaai.v32i1.12255.

[23] P. Hu, N. Xiao, F. Li, Y. Chen, and R. Huang, "A reinforcement learning-based automatic video editing method using pre-trained vision-language model," in Proc. 31st ACM Int. Conf. Multimedia, Ottawa, ON, Canada, 2023, pp. 6441–6450, doi: 10.1145/3581783.3611878.

[24] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," Biometrika, vol. 25, nos. 3–4, pp. 285–294, Dec. 1933, doi: 10.2307/2332286.

[25] E. Kaufmann, N. Korda, and R. Munos, "Thompson sampling: An asymptotically optimal finite-time analysis," in Proc. Int. Conf. Algorithmic Learn. Theory, Lyon, France, 2012, pp. 199–213, doi: 10.1007/978-3-642-34106-9_18.

[26] S. Agrawal and N. Goyal, "Thompson sampling for contextual bandits with linear payoffs," in Proc. 30th Int. Conf. Mach. Learn., Atlanta, GA, USA, 2013, pp. 127–135.

[27] M. Abeille and A. Lazaric, "Linear Thompson sampling revisited," Electron. J. Statist., vol. 11, no. 2, pp. 5165–5197, 2017, doi: 10.1214/17-EJS1341SI.

[28] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, "A tutorial on Thompson sampling," Found. Trends Mach. Learn., vol. 11, no. 1, pp. 1–96, 2018, doi: 10.1561/2200000070.

[29] C. Riquelme, G. Tucker, and J. Snoek, "Deep Bayesian bandits showdown: An empirical comparison of Bayesian deep networks for Thompson sampling," in Proc. Int. Conf. Learn. Represent., Vancouver, BC, Canada, 2018.

[30] D. Zhou, L. Li, and Q. Gu, "Neural contextual bandits with UCB-based exploration," in Proc. 37th Int. Conf. Mach. Learn., 2020, pp. 11492–11502.

[31] D. Bouneffouf, I. Rish, and C. Aggarwal, "Survey on applications of multi-armed and contextual bandits," in Proc. IEEE Congr. Evol. Comput., Glasgow, U.K., 2020, pp. 1–8, doi: 10.1109/CEC48606.2020.9185782.
[32] T. Lattimore and C. Szepesvári, Bandit Algorithms. Cambridge, U.K.: Cambridge Univ. Press, 2020, doi: 10.1017/9781108571401.
[33] Y. Russac, C. Vernade, and O. Cappé, "Weighted linear bandits for non-stationary environments," in Proc. Adv. Neural Inf. Process. Syst., vol. 32, 2019.
[34] B. Kveton et al., "Meta-Thompson sampling," in Proc. 38th Int. Conf. Mach. Learn., 2021, pp. 5884–5893.
[35] L. Cella, A. Lazaric, and M. Pontil, "Meta-learning with stochastic linear bandits," in Proc. 37th Int. Conf. Mach. Learn., 2020, pp. 1360–1370.
[36] J. Hong, B. Kveton, M. Zaheer, Y. Chow, A. Ahmed, and C. Boutilier, "Latent bandits revisited," in Proc. Adv. Neural Inf. Process. Syst., vol. 33, 2020, pp. 13423–13433.
[37] W. B. Knox and P. Stone, "Interactively shaping agents via human reinforcement: The TAMER framework," in Proc. 5th Int. Conf. Knowl. Capture, Redondo Beach, CA, USA, 2009, pp. 9–16, doi: 10.1145/1597735.1597738.
[38] J. MacGlashan et al., "Interactive learning from policy-dependent human feedback," in Proc. 34th Int. Conf. Mach. Learn., Sydney, NSW, Australia, 2017, pp. 2285–2294.
[39] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in Proc. 8th IEEE Int. Conf. Data Mining, Pisa, Italy, 2008, pp. 263–272, doi: 10.1109/ICDM.2008.22.
[40] B. Settles, Active Learning (Synthesis Lectures on Artificial Intelligence and Machine Learning). Cham, Switzerland: Springer, 2012, doi: 10.1007/978-3-031-01560-1.
[41] I. Osband, D. Russo, and B. Van Roy, "(More) efficient reinforcement learning via posterior sampling," in Proc. Adv. Neural Inf. Process. Syst., vol. 26, 2013, pp. 3003–3011.
[42] I. Osband, B. Van Roy, and Z. Wen, "Generalization and exploration via randomized value functions," in Proc. 33rd Int. Conf. Mach. Learn., New York, NY, USA, 2016, pp. 2377–2386.
[43] I. Osband, B. Van Roy, D. J. Russo, and Z. Wen, "Deep exploration via randomized value functions," J. Mach. Learn. Res., vol. 20, no. 124, pp. 1–62, 2019.
[44] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to trust your model: Model-based policy optimization," in Proc. Adv. Neural Inf. Process. Syst., vol. 32, 2019, pp. 12498–12509.
[45] T. Yu et al., "MOPO: Model-based offline policy optimization," in Proc. Adv. Neural Inf. Process. Syst., vol. 33, 2020, pp. 14129–14142.
[46] E. Altman, Constrained Markov Decision Processes. Boca Raton, FL, USA: Chapman & Hall/CRC, 1999.
[47] L. Li, W. Chu, J. Langford, and X. Wang, "Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms," in Proc. 4th ACM Int. Conf. Web Search Data Mining, Hong Kong, 2011, pp. 297–306, doi: 10.1145/1935826.1935878.
[48] Y. Saito and T. Joachims, "Counterfactual learning and evaluation for recommender systems: Foundations, implementations, and recent advances," in Proc. 15th ACM Conf. Recommender Syst., Amsterdam, The Netherlands, 2021, pp. 828–830, doi: 10.1145/3460231.3473320.
[49] D. G. Horvitz and D. J. Thompson, "A generalization of sampling without replacement from a finite universe," J. Amer. Statist. Assoc., vol. 47, no. 260, pp. 663–685, Dec. 1952, doi: 10.1080/01621459.1952.10483446.
[50] A. Swaminathan and T. Joachims, "The self-normalized estimator for counterfactual learning," in Proc. Adv. Neural Inf. Process. Syst., vol. 28, 2015, pp. 3231–3239.
[51] M. Dudík, J. Langford, and L. Li, "Doubly robust policy evaluation and learning," in Proc. 28th Int. Conf. Mach. Learn., Bellevue, WA, USA, 2011, pp. 1097–1104.
[52] A. Swaminathan et al., "Off-policy evaluation for slate recommendation," in Proc. Adv. Neural Inf. Process. Syst., vol. 30, 2017, pp. 3632–3642.
[53] V. Hadad, D. A. Hirshberg, R. Zhan, S. Wager, and S. Athey, "Confidence intervals for policy evaluation in adaptive experiments," Proc. Nat. Acad. Sci. USA, vol. 118, no. 15, 2021, Art. no. e2014602118, doi: 10.1073/pnas.2014602118.
[54] A. C. Cameron, J. B. Gelbach, and D. L. Miller, "Bootstrap-based improvements for inference with clustered errors," Rev. Econ. Statist., vol. 90, no. 3, pp. 414–427, Aug. 2008, doi: 10.1162/rest.90.3.414.
[55] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. 40th Int. Conf. Mach. Learn., Honolulu, HI, USA, 2023, pp. 28492–28518.
[56] G. Smith, D. Yee, J. K. Chen, and L. Findlater, "Prompting Whisper for improved verbatim transcription and end-to-end miscue detection," in Proc. Interspeech, Rotterdam, The Netherlands, 2025, pp. 1943–1947, doi: 10.21437/Interspeech.2025-1031.
[57] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in Proc. Adv. Neural Inf. Process. Syst., vol. 33, 2020, pp. 12449–12460.
[58] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in Proc. 23rd Int. Conf. Mach. Learn., Pittsburgh, PA, USA, 2006, pp. 369–376, doi: 10.1145/1143844.1143891.
[59] M. Bain, J. Huh, T. Han, and A. Zisserman, "WhisperX: Time-accurate speech transcription of long-form audio," in Proc. Interspeech, Dublin, Ireland, 2023, pp. 4489–4493, doi: 10.21437/Interspeech.2023-78.
[60] V. Zayats, M. Ostendorf, and H. Hajishirzi, "Disfluency detection using a bidirectional LSTM," in Proc. Interspeech, San Francisco, CA, USA, 2016, pp. 2523–2527, doi: 10.21437/Interspeech.2016-1247.
[61] M. Honnibal and M. Johnson, "Joint incremental disfluency detection and dependency parsing," Trans. Assoc. Comput. Linguistics, vol. 2, pp. 131–142, 2014, doi: 10.1162/tacl_a_00171.
[62] P. J. Lou and M. Johnson, "End-to-end speech recognition and disfluency removal," in Proc. Findings Assoc. Comput. Linguistics: EMNLP, 2020, pp. 2051–2061, doi: 10.18653/v1/2020.findings-emnlp.186.
[63] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods Natural Lang. Process., Hong Kong, 2019, pp. 3980–3990, doi: 10.18653/v1/D19-1410.
[64] W. Wu, H. Peng, and S. Yu, "YuNet: A tiny millisecond-level face detector," Mach. Intell. Res., vol. 20, no. 5, pp. 656–665, Oct. 2023, doi: 10.1007/s11633-023-1423-y.